Rapport de stage de fin d'année

ISTY – IATIC4 – Année universitaire 2025–2026

---

calculs de similarité entre exercices de programmation informatique

pour la recommandation et la substitution d'exercices dans les EIAH

Juvenal MALECOU NGUIMBI

Structure d'accueil :	Laboratoire DICEN-IdF – Université Gustave Eiffel
Lieu :	Campus de Marne-la-Vallée, 5 bd Descartes, 77454 Champs-sur-Marne
Dates du stage :	du 08/06/2026 au 05/11/2026
Maître de stage :	M. Olivier CHAMPALLE
Tuteur pédagogique (ISTY) :	Mme Laurence PILARD
Confidentialité :	OUI

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur Olivier Champalle, mon maître de stage au sein du laboratoire DICEN-IdF, pour la confiance qu'il m'a accordée en m'intégrant à ce projet de recherche. Ses conseils avisés, sa disponibilité et son encadrement scientifique ont été déterminants dans la structuration de ma démarche et m'ont permis d'approfondir considérablement mes connaissances sur le sujet.

Mes remerciements s'adressent également à Madame Laurence Pilard, ma tutrice pédagogique à l'ISTY, pour son suivi attentif tout au long de cette période, ses conseils et son accompagnement.

Je souhaite aussi exprimer ma gratitude envers Adel Ziani, dont les travaux préliminaires ont constitué un point de départ précieux pour mes propres expérimentations.

Je remercie plus largement l'ensemble des membres de l'équipe Platon et de l'Université Gustave Eiffel pour leur accueil et pour avoir créé un environnement de travail à la fois stimulant et convivial au quotidien.

Enfin, je remercie l'équipe pédagogique de l'ISTY pour la qualité de la formation d'ingénieur dispensée, ainsi que ma famille et mes proches pour leur soutien indéfectible tout au long de mes études.

Résumé

Ce rapport étudie la substituabilité entre exercices de programmation, c'est-à-dire la possibilité d'identifier des activités qui, sans être identiques, mobilisent des connaissances, des structures de contrôle et des démarches algorithmiques suffisamment proches pour constituer des alternatives pertinentes.

Les objectifs sont multiples : aider un enseignant à retrouver, dans une banque d'exercices, une activité proche d'un problème existant ; recommander à un étudiant un entraînement sollicitant une démarche comparable, sans lui faire répéter exactement la même consigne,etc... La substituabilité ne doit toutefois pas être confondue avec la similarité textuelle, la détection de plagiat ou l'équivalence fonctionnelle de deux programmes.Bien que les methodes utilisée sont partagée entre elle Deux exercices peuvent traiter des problèmes différents tout en mobilisant des stratégies de résolution proches. À l'inverse, deux énoncés utilisant un vocabulaire semblable peuvent nécessiter des démarches distinctes.

Dans le contexte étudié, aucune paire d'exercices n'est annotée selon son degré de substituabilité et aucune trace d'activité ou de performance d'apprenants n'est disponible. L'étude s'appuie donc principalement sur les propriétés intrinsèques des corrections de référence. Des expérimentations préliminaires sur les énoncés, fondées sur TF–IDF et SBERT, ont fourni un signal de proximité thématique. Les ambiguïtés, la variabilité des formulations et la dépendance de certaines consignes à un contexte implicite conduisent cependant à retenir le code comme source principale d'information.

Parmi la diversité des approches qui existent sur des sujets parralles qui se font dans la literature ,trois on retenu notre attention et par les quels on va pouvoir extraire trois carractéristique issue des corrections : une similarité par distance d'édition entre arbres syntaxiques abstraits normalisés (TSED), une similarité globale fondée sur des métriques structurelles (EIIa), et une similarité calculée à partir d'un *Normalized Control Structure Tree* (NCST)qui est une construction de notre part. Construit à partir de l'arbre syntaxique concret produit par Tree-sitter(un outil de génération d'analyseurs et une bibliothèque d'analyse incrémentielle ou parseur simplement), le NCST est normalisé afin de conserver l'organisation des structures de contrôle, notamment les conditions, les itérations et les transferts, tout en réduisant l'influence des détails syntaxiques et des styles d'écriture.

Les méthodes sont évaluées sur des variantes contrôlées de programmes, puis sur un corpus plus large. Elles produisent des indicateurs continus et interprétables permettant de filtrer des paires candidates à une validation pédagogique humaine, nécessaire pour déterminer si la proximité structurelle observée correspond effectivement à une substituabilité du point de vue de l'apprentissage.

Mots-clés : EIAH,similarité de code,normalisation de code, exercices de programmation, EIAH, arbre syntaxique abstrait, distance d'édition, structures de contrôle, normalisation de code.

Abstract

This report studies the substitutability between programming exercises, that is, the possibility of identifying activities that, without being identical, involve sufficiently similar knowledge, control structures, and algorithmic approaches to constitute relevant alternatives.

The objectives are multiple : to help a teacher find, in an exercise bank, an activity close to an existing problem ; to recommend to a student a training exercise involving a comparable approach, without making them repeat exactly the same problem, etc. Substitutability should not, however, be confused with textual similarity, plagiarism detection, or the functional equivalence of two programs. Although the methods used share some common aspects, two exercises can address different problems while mobilizing similar solution strategies. Conversely, two exercise statements using similar vocabulary may require distinct approaches.

In the context studied, no pair of exercises is annotated according to its degree of substitutability, and no learner activity or performance data are available. The study therefore relies mainly on the intrinsic properties of the reference solutions. Preliminary experiments on the exercise statements, based on TF–IDF and SBERT, provided a signal of thematic proximity. However, the ambiguities, variability of formulations, and dependence of some statements on implicit context lead us to use the code as the main source of information.

Among the diversity of approaches found in the literature on related topics, three approaches caught our attention, through which we can extract three characteristics from the reference solutions : a similarity measure based on the edit distance between normalized abstract syntax trees (TSED), a global similarity measure based on structural metrics (EIIa), and a similarity measure calculated from a **Normalized Control Structure Tree** (NCST), which is a construction developed as part of our work.

The NCST is constructed from the concrete syntax tree produced by Tree-sitter (a tool for generating parsers and an incremental parsing library, or more simply, a parser). The NCST is normalized in order to preserve the organization of the control structures, particularly conditions, iterations, and transfers, while reducing the influence of syntactic details and programming styles.

The methods are evaluated on controlled variants of programs, and then on a larger corpus. They produce continuous and interpretable indicators that make it possible to filter candidate pairs for human pedagogical validation, which is necessary to determine whether the observed structural proximity effectively corresponds to substituability from a learning perspective.

Keywords : EIAH, code similarity, code normalization, programming exercises, abstract syntax tree, edit distance, control structures.

Table des matières

Remerciements **i**

Résumé **ii**

Lexique **x**

Introduction générale **1**

1 Présentation des structures d'accueil **2**

1.1 L'Université Gustave Eiffel 2

1.2 Le laboratoire DICEN-IdF 3

1.3 L'institut Gaspard Monge 3

2 Contexte et organisation du stage **4**

2.1 Descriptifs du projet 4

2.2 Logistique du stage 6

2.3 Méthodologie 7

3 État de l'art et positionnement **8**

3.1 Qu'est-ce que la similarité ? 8

3.2 Domaine et approches des travaux sur les similarités dans les EIAH 10

3.3 Quels sont les outils et méthodes utilisés par ces approches ? 12

3.4 Discussion sur les limites des méthodes 18

3.5 Pistes explorées et écartées 20

4 Ma proposition **25**

4.1 Architecture globale du pipeline 25

4.2 Évaluation de l'information indépendante de l'arbre (EIIA) 27

4.3 AST normalisé et distance TSED 28

4.3.1 Conversion du CST en AST simplifié 28

4.3.2 Normalisation de l'AST 29

4.3.3 Distance d'édition entre arbres . . . 30
4.4 Arbre structurel normalisé (NCST) . . . 31
4.4.1 Construction et normalisation du NCST . . . 31
4.4.2 Niveau structurel . . . 32
4.4.3 Exemple de NCST normalisé et linéarisation . . . 32
4.4.4 Invariance du squelette structurel . . . 33
4.4.5 Algorithme d'appariement naïf ordonné . . . 34
4.4.6 Calcul de la similarité NCST . . . 35
4.4.7 Positionnement par rapport au TSED et à l'EIIA . . . 36
4.4.8 Limites de l'approche . . . 36
4.5 Exécution et synthèse des mesures . . . 36
4.6 Interface de comparaison Streamlit . . . 37
4.6.1 Préparation des codes à comparer . . . 37
4.6.2 Filtrage des fichiers syntaxiquement invalides . . . 38
4.6.3 Exécution et pondération des scores . . . 38
4.6.4 Restitution des résultats . . . 39
4.7 Protocole d'évaluation des approches . . . 39

5 Résultats expérimentaux **41**
5.1 Constitution du corpus et variantes contrôlées . . . 41
5.2 Résultats sur les variantes contrôlées . . . 42
5.2.1 EIIA : attendus et observés . . . 44
5.2.2 TSED : attendus et observés . . . 44
5.2.3 NCST : attendus et observés . . . 45
5.2.4 Bilan sur les variantes contrôlées . . . 46
5.2.5 Fusion des trois approches . . . 46
5.3 Résultats sur le corpus complet . . . 46
5.4 Synthèse comparative . . . 46

6 Discussion sur les limites et perspectives **47**
6.1 Discussion des résultats . . . 47
6.2 Limites techniques des méthodes . . . 47
6.3 Limites de l'évaluation . . . 47
6.4 Difficultés rencontrées . . . 47
6.5 perspective . . . 47

Conclusion générale **48**

Bibliographie **49**

A Annexes **52**

Table des figures

1.1 Regroupement des établissements formant l'Université Gustave Eiffel 2

2.1 Acteurs du stage et environnement de travail 6
2.2 Diagramme de Gantt du stage 7
2.3 Démarche de travail suivie pendant le stage 7

3.1 Les quatre grandes finalités de la mesure de similarité dans les EIAH (orientations divergentes à partir d'un même besoin technique). 10
3.2 Étapes communes aux méthodes de comparaison d'exercices pédagogiques 12
3.3 Familles de représentations mobilisées pour comparer des exercices de programmation 17
3.4 Cartographie des limites : pourquoi chaque approche individuelle échoue à caractériser seule la substituabilité pédagogique. 19
3.5 Comparaison des scores de similarité cosinus TF–IDF et SBERT pour l'ensemble des paires d'énoncés. La ligne pointillée représente l'égalité des scores. 21

4.1 Architecture globale du pipeline d'évaluation de la similarité multi-niveaux. L'EIIA et le TSED partagent l'AST normalisé ; le NCST est construit directement à partir du CST. L'interface Streamlit exploite le CSV produit par le pipeline. 26
4.2 Chaîne de calcul de l'EIIA : extraction de deux profils structurels, puis distance euclidienne normalisée. 28
4.3 Suppression des nœuds non reconnus, marqués `(x)` : leurs enfants prennent leur place dans le même ordre, et les instructions voisines ne sont pas déplacées. Le nœud `as_pattern_target` qui entoure `fh` et `g` est omis de l'extrait ; il est supprimé de la même manière. 29
4.4 Normalisation d'une instruction. Les nœuds en rouge portent des détails de surface (noms, valeurs, sucre syntaxique) ; après normalisation, ils sont remplacés par des formes canoniques (en vert). Les opérateurs sont conservés. 30
4.5 Exemple de calcul TSED sur deux AST normalisés de quatre nœuds. 31
4.6 Calcul du niveau structurel dans le NCST : seules les arêtes qui ouvrent un bloc incrémentent le niveau. 32
4.7 Deux imbrications inverses conduisent à deux NCST normalisés différents (niveaux relatifs, la racine du fragment étant au niveau 0). L'appariement de la section 4.4.5 n'associe que `SEQUENCE`, `SEQUENCE` et `STATEMENT` : $M = 3$ et $S_{\text{NCST}} = 6/10 = 0{,}6$. . 33

4.8 NCST normalisé d'une fonction (gauche), avec entre crochets la relation qui relie chaque nœud à son parent, et liste (*kind*, *level*) obtenue par parcours en profondeur (droite). Le retrait traduit la profondeur dans l'arbre, tandis que le niveau n'augmente que sur les arêtes `BODY` et `THEN`. 33

4.9 Invariance du NCST normalisé sur deux programmes différents. Les codes A et B utilisent des variables, des opérateurs et des valeurs de retour distincts, mais leur normalisation produit la même liste de nœuds abstraits. 34

4.10 Appariement ordonné du petit arbre vers le grand arbre. Les pointillés relient des nœuds de même *kind* et de même *level*, dans un ordre croissant d'indices. 35

4.11 Fonctionnement de l'interface Streamlit : préparation du jeu de fichiers, appel du pipeline, pondération facultative et restitution des scores et des arbres. 37

4.12 Protocole d'évaluation en deux phases. 40

5.1 Correction de référence et variante contrôlée « `while` avec index » : même stratégie, structure de parcours différente. 42

Liste des tableaux

4.1 Synthèse des trois niveaux de représentation du pipeline. 26
4.2 Composantes du profil structurel utilisé par l'EIIA. 27
4.3 Règles de conversion du CST Tree-sitter en AST interne. Aucun nœud `UNKNOWN` n'est produit. 30
4.4 Vocabulaire du NCST normalisé. 32
4.5 Structure du fichier CSV exporté par le pipeline : une ligne par paire, une colonne par mesure, et une colonne d'erreur en cas d'échec de l'analyse. 36
4.6 Exemples de profils de scores et de leur interprétation. 37
4.7 Onglets d'inspection proposés pour chaque fichier source. 39

5.1 Statistiques descriptives du corpus de corrections de référence. 42
5.2 Statistiques descriptives du jeu de variantes contrôlées. 42
5.3 Variantes contrôlées dérivées de la correction de référence. 43
5.4 Scores obtenus entre la correction de référence `01_base.py` et chacune de ses douze variantes contrôlées. 43

Lexique

Les termes sont classés par ordre alphabétique. Les sigles anglais sont conservés lorsqu'ils sont d'usage dans la littérature.

Apprenant
Personne qui suit une formation ; dans ce rapport, l'étudiant qui réalise les exercices de programmation.

AST (*Abstract Syntax Tree*, arbre syntaxique abstrait)
Représentation arborescente d'un programme dans laquelle chaque nœud correspond à une construction du langage (boucle, condition, affectation, appel de fonction), indépendamment de la mise en forme du texte.

CFG (*Control Flow Graph*, graphe de flot de contrôle)
Graphe dont les nœuds sont des instructions et dont les arcs indiquent l'ordre possible d'exécution.

Clone de code
Fragment de code identique ou très proche d'un autre fragment ; la détection de clones cherche à les repérer dans un logiciel.

CMGA
Méthode de détection de code sémantiquement similaire combinant un filtrage par métriques et une comparaison de graphes de dépendances normalisés [29].

Cold start (démarrage à froid)
Situation dans laquelle un système doit fonctionner sans données historiques (annotations, résultats d'apprenants) sur lesquelles s'appuyer.

Corpus
Ensemble des exercices (énoncés et corrections) utilisés pour les expérimentations.

CPG (*Code Property Graph*)
Graphe unique, produit par l'outil Joern, qui superpose l'AST, le CFG et les dépendances d'un programme.

CSSG
Mesure de similarité de code fondée sur des graphes sémantiques [15].

Distance d'édition
Coût minimal des opérations élémentaires (insertion, suppression, substitution) nécessaires pour transformer un objet en un autre ; s'applique aux chaînes, aux arbres (TED) et aux graphes (GED).

EIAH
Environnement informatique pour l'apprentissage humain : logiciel ou plateforme conçu pour

soutenir l'apprentissage, par exemple un exerciseur de programmation.

Embedding (plongement vectoriel)
Représentation d'un texte ou d'un code sous forme de vecteur de nombres, telle que deux éléments proches en sens soient proches dans l'espace vectoriel.

Exerciseur
EIAH proposant des exercices à correction automatique pour l'entraînement.

GED (*Graph Edit Distance*)
Distance d'édition entre graphes.

Joern
Outil d'analyse statique de code, multi-langage, produisant un Code Property Graph.

Normalisation de code
Ensemble de transformations qui ramènent des écritures différentes mais équivalentes d'un programme à une forme commune (renommage canonique, unification des boucles, suppression des commentaires, etc.).

NCST
Normalized Control Structure Tree : arbre normalisé des structures de contrôle, construit à partir du CST Tree-sitter et utilisé pour comparer l'organisation des fonctions, séquences, boucles, conditions, exceptions et transferts de contrôle.

EIIa
Indicateur global calculé à partir du code source, utilisé pour estimer et comparer la complexité structurelle de deux corrections.structurelles extraites des AST normalisés.

PDG (*Program Dependence Graph*)
Graphe qui réunit, pour une fonction, les dépendances de contrôle et les dépendances de données entre instructions.

Recommandation d'exercices
Proposition automatique à un apprenant d'un exercice adapté à sa progression.

Transformers biencoder

SBERT (*Sentence-BERT*)
Modèle de langage qui transforme une phrase en vecteur afin de comparer des phrases par leur sens [25].

Substituabilité
Propriété de deux exercices qui peuvent être échangés dans un parcours d'apprentissage parce qu'ils mobilisent des notions et une démarche de résolution proches.

TED (*Tree Edit Distance*)
Distance d'édition entre arbres ordonnés ; l'algorithme de Zhang et Shasha en est le calcul de référence [31].

Token (lexème)
Unité lexicale élémentaire d'un programme (mot-clé, identifiant, opérateur, littéral) produite par

l'analyse lexicale.

Vectorisation

Embedding

TSED
Similarité de code obtenue en normalisant la distance d'édition entre AST [26].

Variante contrôlée
Version d'un programme de référence obtenue par une transformation unique et identifiée (renommage, changement de boucle, extraction de fonction), utilisée pour observer la réaction d'une mesure à cette transformation.

Variation de code
Phénomène par lequel une même tâche peut être programmée de nombreuses façons syntaxiquement différentes mais sémantiquement équivalentes [29].

Vérité terrain
Référence considérée comme exacte (par exemple un jugement d'enseignant) à laquelle on compare les résultats d'une méthode automatique.

Introduction générale

[À compléter : 2 pages]

Chapitre 1

Présentation des structures d'accueil

Dans ce chapitre, je présente l'Université Gustave Eiffel, au sein de laquelle s'est déroulé mon stage, le laboratoire DICEN-IdF, qui a financé ce stage, ainsi que l'Institut Gaspard Monge (IGM), où j'ai été en etroite colaboration avec l'equipe Platon qui sont en charge de l'application ou vas principalement s'appliqué les resultats des recherches.

1.1 L'Université Gustave Eiffel

L'Université Gustave Eiffel est une université française créée en janvier 2020 par la fusion de plusieurs établissements (figure 1.1) :

- **L'UPEM** : L'Université Paris-Est Marne-la-Vallée, fondée en 1991.
- **L'IFSTTAR** : L'Institut français des sciences et technologies des transports, de l'aménagement et des réseaux.
- **L'ESIEE Paris** : Une école d'ingénieurs située à la cité Descartes.
- **L'ENSG-Géomatique** : Une école d'ingénieurs spécialisée en géomatique.
- **L'EIVP** : Une école d'ingénieurs située à Paris.
- **L'Éav&t** : L'école d'architecture de la ville et des territoires Paris-Est.

L'UGE contient au total 16 composantes d'enseignement ainsi que 23 composantes de recherche. Parmi ses principales activités, on retrouve la formation, la recherche et l'innovation.

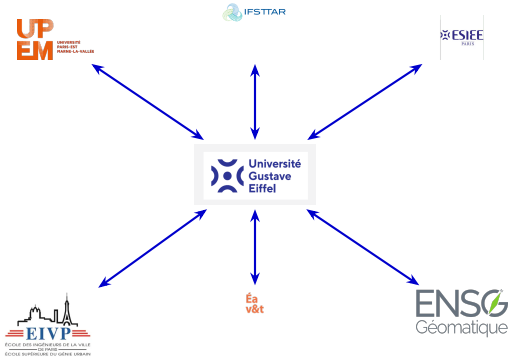


Figure 1.1 – Regroupement des établissements formant l'Université Gustave Eiffel

1.2 Le laboratoire DICEN-IdF

Le laboratoire DICEN-IdF (Dispositifs d'Information et de Communication à l'Ère Numérique – Paris Île-de-France) est une unité de recherche multi-tutelles. Ses axes de recherche portent sur l'ingénierie documentaire, les humanités numériques et l'intelligence collective.

C'est à ce laboratoire qu'est rattaché mon maître de stage, Monsieur Olivier Champalle, et c'est également le DICEN-IdF qui finance ce stage . À ce titre, le laboratoire constitue le cadre scientifique principal du projet, en y apportant les fonds nécessaires ainsi que son expertise sur l'ingénierie des connaissances et l'analyse des traces d'apprentissage dans les environnements numériques.

1.3 L'institut Gaspard Monge

Bien que mon stage soit financé par le DICEN-IdF, il s'est déroulé physiquement dans les locaux de l'Institut Gaspard Monge (L'IGM), situés sur le campus de l'Université Gustave Eiffel.

L'IGMest un institut reconnu pour ses travaux en informatique fondamentale et appliquée. C'est sur ce site que j'ai mené mes recherches au quotidien, ce qui m'a permis de travailler en étroite collaboration avec l'équipe Platon. Platon est un exerciseur – développé et géré par cette équipe. Cette proximité géographique et technique était essentielle, car c'est au sein de cette plateforme pédagogique que les algorithmes de calcul de similarité ont vocation à être intégrés et exploités principalement.

Chapitre 2

Contexte et organisation du stage

Dans ce chapitre, je présente le projet dans lequel s'inscrit mon stage, puis la manière dont le stage a été organisé, et enfin la démarche qu'on a suivie pour mener ce travail de recherche.

2.1 Descriptifs du projet

Dans le cadre des enseignements d'algorithmique et de programmation informatique, les enseignants s'appuient de plus en plus sur des Environnements Informatiques pour l'Apprentissage Humain (EIAH), tels que la plateforme PLaTon. La base de cette plateforme est constituée d'une multitude d'exercices, chacun étant défini par un énoncé textuel et une correction de référence. L'idée fondatrice du projet est de comprendre s'il est possible d'identifier des exercices « proches » ou « distants » entre eux, quel que soit le langage de programmation, et de caractériser cette distance de manière à construire une classification réutilisable.

Cette question est complexe et soulève des enjeux sémantiques. Si la langue française dispose de nombreux termes pour l'exprimer (deux exercices peuvent être *analogues*, *comparables*, *semblables*, *voisins*, *équivalents* ou *substituables*), chaque mot porte une nuance différente. Pour évaluer la substituabilité pédagogique, plusieurs critères ont été identifiés :

— La **mobilisation des mêmes connaissances**, comme l'utilisation d'un même type de structure de données (liste, dictionnaire) ou d'une même notion algorithmique.
— Un **comportement** et une **complexité** similaires, tant dans l'architecture du code que dans le temps d'exécution.
— Des **résultats pédagogiques** similaires, garantissant que le travail de l'apprenant cible bien le même concept, même si le résultat brut de l'exécution diffère.
— Une **difficulté** (ou un effort de réalisation) comparable pour l'étudiant.

La classification recherchée doit prendre en compte : les concepts et notions d'algorithmique nécessaires pour résoudre l'exercice, et les compétences propres au langage de programmation utilisé (Python, Java, C, etc.), avec leur niveau de difficultés. La définition précise de la similarité est approfondie dans l'état de l'art.

Une telle mesure de similarité ouvre plusieurs cas d'utilisation, à plus ou moins long terme, au sein des EIAH de type exerciseur.

Pour l'enseignant :

— Rechercher rapidement dans la base un exercice similaire à ceux qu'il possède déjà dans ses fiches de TP, afin d'éviter la redondance.
— Se voir suggérer des variantes « proches » structurellement pour préparer des évaluations.
— Repartir d'un exercice existant pour le dériver à la marge.
— Catégoriser et étiqueter automatiquement la difficulté et les notions mobilisées dans la base d'exercices.

Pour l'étudiant :

— S'entraîner sur des exercices proches mais non identiques, afin de renforcer des notions sans refaire la même consigne.
— Bénéficier d'un système de recommandation adaptatif suggérant des exercices de difficulté équivalente.

Pour l'évaluation des IA :

— Faciliter la validation (ou l'invalidation) d'exercices générés par une intelligence artificielle, en comparant automatiquement leur structure avec le corpus validé par les enseignants.

Le projet fait face à une contrainte majeure dite de *démarrage à froid* (*cold start*). Aucune paire d'exercices n'est annotée formellement pour indiquer son degré de substituabilité, et aucune trace d'activité d'apprenants (temps passé, taux d'erreur) n'est disponible. L'analyse doit donc s'opérer de manière purement statique et non supervisée, en se fondant exclusivement sur les énoncés et l'unique correction de référence de chaque exercice.

Ainsi l'objectif est de proposer des mesures automatiques de similarité entre exercices, utilisables sans données d'apprenants et interprétables par un enseignant, afin d'identifier des paires d'exercices candidates à la substitution. Pour y parvenir, les missions qui m'ont été confiées sont les suivantes :

— Réaliser un état de l'art sur la notion de similarité entre exercices, algorithmes et codes.
— M'appuyer sur le travail préliminaire d'Adel Ziani (stage de M1, 2025) [32], qui a permis de construire la base de données et d'explorer les premières approches textuelles.
— Créer, adapter et implémenter plusieurs méthodes de détection de similarité.
— Valider et calibrer ces méthodes sur des données expérimentales.

Les livrables attendus sont le code source des algorithmes , les jeux de données expérimentaux, ainsi que le présent rapport, qui comprend l'état de l'art.

2.2 Logistique du stage

Ce stage, d'une durée d'environs cinq mois, s'est déroulé du 8 juin au 5 novembre 2026. Bien que rattaché au laboratoire DICEN-IdF (Université Gustave Eiffel), le travail quotidien a été effectué dans les locaux de l'Institut Gaspard Monge (IGM), sur le campus de Marne-la-Vallée à Champs-sur-Marne.

L'encadrement scientifique a été assuré par mon maître de stage, Monsieur Olivier Champalle (DICEN-IdF), à l'origine du sujet de recherche. Au quotidien, j'ai travaillé à proximité de l'équipe PLaTon, ce qui me permettait d'échanger sur le fonctionnement de la plateforme et sur la structure des exercices. Le suivi pédagogique, garantissant l'adéquation du stage avec les exigences de ma formation à l'ISTY, a été assuré par Madame Laurence Pilard. La figure 2.1 résume l'organisation des acteurs autour du projet.

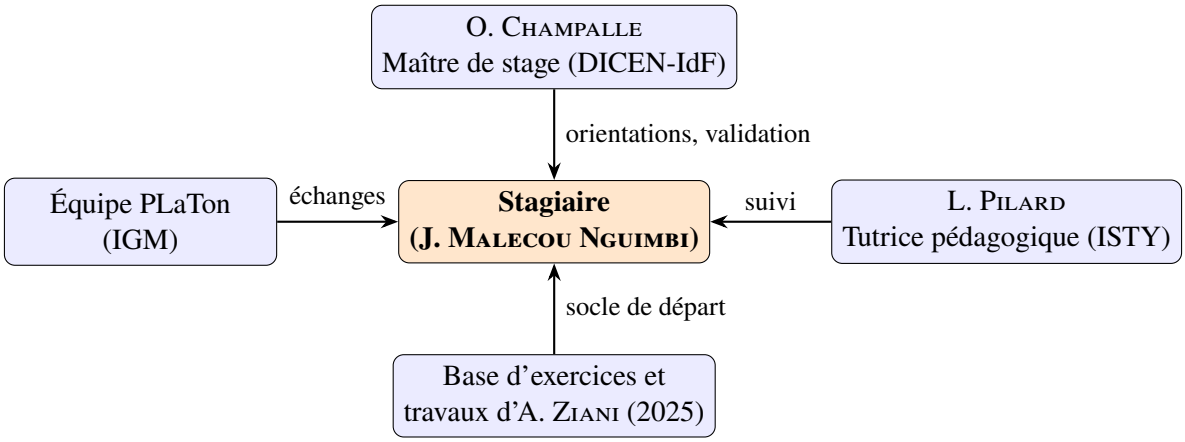


Figure 2.1 – Acteurs du stage et environnement de travail

Des points réguliers ont eu lieu avec mon maître de stage pour discuter des expérimentations, valider les choix techniques, comme le passage d'une similarité fondée sur les énoncés à une similarité fondée sur le code, et planifier la suite du travail.

- **Python** : langage principal pour l'ensemble des scripts d'expérimentation ;
- **Tree-sitter** : outil d'analyse syntaxique (*parser*) utilisé pour analyser le code source et produire des arbres syntaxiques, pour de nombreux langages de programmation ;
- **`scikit-learn` et `sentence-transformers`** : bibliothèques utilisées pour les expériences sur les énoncés (TF-IDF et SBERT) ;
- **Git** : gestion des versions du code ;
- **Google Scholar** : moteur de recherche d'articles scientifiques, utilisé pour l'état de l'art ;
- **Zotero** : stockage des papiers de recherche et rédaction des résumés ;
- **Google Drive** : partage des documents, des résultats d'expérimentation et des comptes rendus avec mon maître de stage ;
- **LaTeX** : rédaction du rapport.

La figure 2.2 présente le déroulement du stage. Les phases situées à gauche de la ligne « Aujourd'hui » sont réalisées ; les phases à droite correspondent au travail restant : l'évaluation sur le corpus complet et la finalisation du rapport.

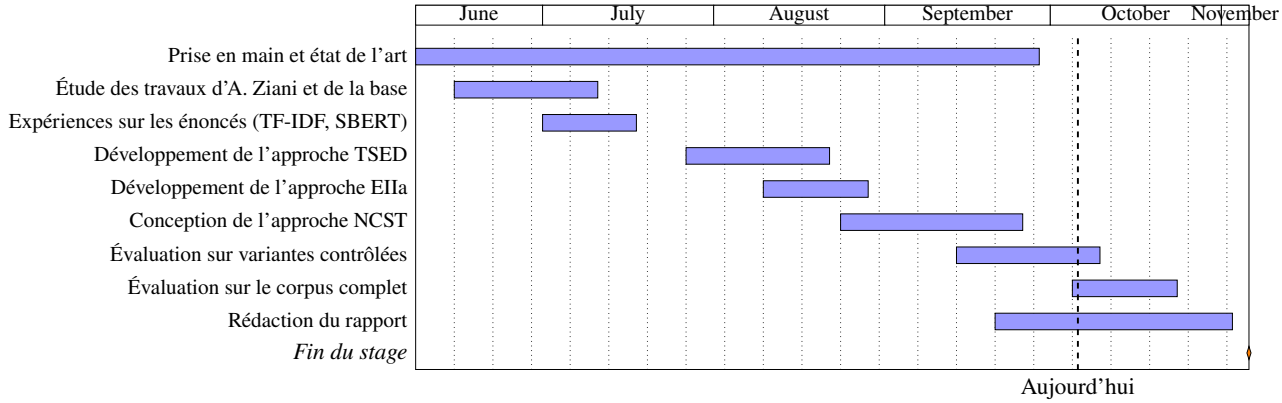


FIGURE 2.2 – Diagramme de Gantt du stage

2.3 Méthodologie

Ce stage étant un travail de recherche, la démarche n'était pas entièrement fixée à l'avance : elle s'est construite par étapes, en ajustant la direction en fonction des résultats obtenus. Elle suit les trois attendus du sujet (état de l'art, méthodes, validation) et peut se résumer en cinq étapes (figure 2.3) :

1. **Étude bibliographique** : lire les travaux existants sur la similarité entre exercices, algorithmes et codes, afin de comprendre les approches possibles et leurs limites.
2. **Exploration** : reprendre la base d'exercices et les travaux d'A. ZIANI, puis tester rapidement plusieurs pistes, comme la comparaison des énoncés par TF-IDF et SBERT, afin d'identifier celles qui donnent un signal utile. Les ambiguïtés et la variabilité des énoncés ont conduit à retenir le code des corrections comme source principale d'information.
3. **Conception et implémentation** : développer les trois approches retenues, EIIa, TSED et NCST.
4. **Validation et calibration** : vérifier le comportement de chaque approche, d'abord sur des variantes contrôlées, c'est-à-dire des programmes modifiés volontairement d'une seule façon, puis sur un corpus plus large.
5. **Analyse et rédaction** : interpréter les résultats, identifier les limites et rédiger le rapport.

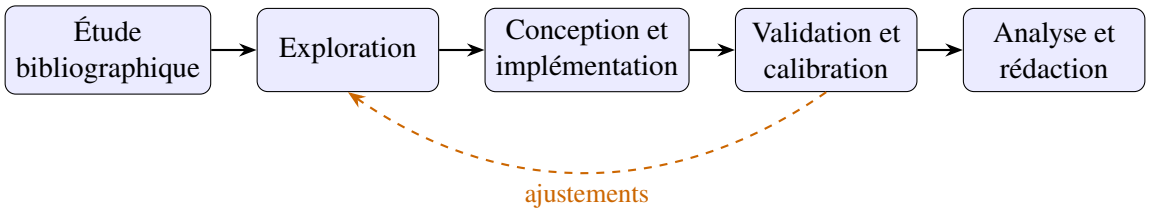


FIGURE 2.3 – Démarche de travail suivie pendant le stage

Cette démarche est itérative : les résultats de la validation ont souvent conduit à revenir sur une approche pour la corriger ou l'améliorer.

Chapitre 3

État de l'art et positionnement

Ce chapitre dresse un panorama des recherches actuelles concernant l'analyse de code source, particulièrement dans un contexte éducatif. Avant de détailler l'architecture de notre propre pipeline d'analyse, il est essentiel de comprendre comment le concept même de similarité est défini et mesuré par la communauté scientifique.

Dans un premier temps, nous définirons formellement la notion de similarité appliquée similarité entre exercices , en distinguant les différentes manières de l'appréhender en informatique. Nous explorerons ensuite les domaines d'application et les principales approches classiques utilisées dans les Environnements Informatiques d'Apprentissage Humain (EIAH). Nous détaillerons par la suite les outils techniques et les méthodes spécifiques mobilisés par ces travaux, tels que les Arbres de Syntaxe Abstraite (AST) ou les métriques de distance. Enfin, une discussion critique mettra en évidence les limites de ces méthodes existantes. Cette analyse nous permettra de justifier notre positionnement scientifique et de motiver les choix techniques qui constituent le cœur de notre proposition, présentée dans le chapitre suivant.

3.1 Qu'est-ce que la similarité ?

En informatique, et plus particulièrement dans l'évaluation d'exercices de programmation, la similarité désigne la mesure du degré de ressemblance entre deux fragments de programme. Toutefois, dans la littérature scientifique, cette notion n'est pas un concept univoque ni une propriété absolue. Comme le souligne Pelánek [22] dans sa revue systématique, il n'existe pas de mesure universelle : la définition même de ce qui constitue la « proximité » est une construction conceptuelle qui varie radicalement selon le problème à résoudre et les données disponibles.

Concrètement, les chercheurs considèrent que deux codes sont similaires selon trois grands paradigmes :

— **La similarité par l'origine (l'intention de copie) :** Dans les travaux liés à la détection de plagiat, deux codes sont considérés comme similaires s'ils partagent une origine commune trahissant une copie. La similarité se mesure à la quantité de particularités d'écriture communes (erreurs identiques, style de formatage, choix de variables atypiques), et ce, même si l'un des programmes a subi des transformations de surface (offuscation, renommage) pour masquer cette filiation. Cette conception est notamment illustrée par JPlag, qui compare des programmes d'étudiants afin d'identifier des cas de plagiat [23]. Les travaux fondés sur les arbres syntaxiques abstraits étendent également cette idée en recherchant des clones structurellement proches dans le code

[3].

— **La similarité par l'équivalence (l'approche formelle) :** D'un point de vue plus théorique, certains travaux définissent la similarité comme une stricte équivalence comportementale. Deux programmes sont jugés similaires s'ils calculent la même fonction mathématique (ils produisent les mêmes sorties pour les mêmes entrées). Cette approche cherche à abstraire totalement le code pour le traduire en modèles formels. La vérification formelle de l'équivalence de programmes est notamment étudiée par [17]. Dans le contexte de l'évaluation d'exercices, des vérificateurs formels peuvent comparer les soumissions des étudiants aux solutions de référence [18]. Si l'équivalence est prouvée, les codes sont déclarés similaires, quand bien même leurs syntaxes n'auraient rien en commun.

— **La similarité par les caractéristiques (l'approche structurelle) :** Enfin, une vision pragmatique considère que la similarité repose sur le partage de caractéristiques mesurables de résolution. Deux codes sont évalués comme proches s'ils affichent des métriques logicielles comparables ou s'ils s'appuient sur un squelette algorithmique analogue (même enchaînement et imbrication des structures de contrôle). Ces caractéristiques peuvent être extraites à partir d'arbres syntaxiques abstraits et de distances d'édition [3, 31, 26], de graphes de flot de contrôle [28] ou de graphes de dépendances [30, 24]. Les approches combinant des métriques et des graphes normalisés cherchent en outre à détecter des programmes proches malgré les variations de syntaxe et de style [29, 15].

Dans le domaine des EIAH, l'objectif n'est ni de traquer le plagiat, ni de prouver une équivalence mathématique stricte. La similarité vise à identifier des activités susceptibles d'être rapprochées pour la recherche, la recommandation ou la personnalisation des parcours d'apprentissage [14, 27]. Elle ne se réduit donc pas à une simple ressemblance textuelle entre les énoncés : deux exercices peuvent être formulés avec des mots très différents tout en mobilisant une démarche algorithmique identique, et inversement.

Dans le contexte de ce rapport, nous définissons la similarité entre deux exercices comme le **degré de proximité entre les caractéristiques pédagogiques et structurelles de leurs solutions de référence**. Ces caractéristiques concernent l'organisation des instructions, les structures de contrôle utilisées, les notions algorithmiques mobilisées et la complexité du traitement. Pour les capturer, l'analyse s'appuie sur des représentations telles que les arbres syntaxiques abstraits (AST) [3], les distances d'édition entre arbres [31] et une representation issue de Wang *et al.* [29]

On peut formaliser cette idée par une fonction de similarité S associant à deux exercices e_i et e_j un score continu :

$$S(e_i, e_j) \in [0, 1]$$

Une valeur proche de 1 indique une forte proximité algorithmique (stratégies de résolution analogues), tandis qu'une valeur proche de 0 signale des approches fondamentalement distinctes.

Ainsi, la similarité constitue dans ce travail un **indicateur multidimensionnel**. Estimée automatiquement à partir des propriétés intrinsèques du code de l'enseignant, cette valeur ne constitue pas à elle seule un jugement pédagogique définitif. Une forte similarité structurelle permet de filtrer des paires candidates et de signaler que deux exercices sont potentiellement substituables, mais cette substituabilité doit au final être confirmée par une analyse humaine tenant compte des notions visées et des objectifs d'apprentissage.

3.2 Domaine et approches des travaux sur les similarités dans les EIAH

La mesure de similarité entre exercices constitue un problème important dans les Environnements Informatiques pour l'Apprentissage Humain (EIAH). Elle intervient dans plusieurs tâches : l'évaluation automatique des productions, la détection de plagiat ou de clones, la recherche de ressources, la recommandation pédagogique,assister l'apprentissage de la programmation. et la personnalisation des parcours d'apprentissage. Ces tâches ont **un objectif commun – « rapprocher des productions ou des activités »** – mais ne donnent pas le même **sens** à la notion de similarité. Une méthode adaptée à la détection d'une copie n'est donc pas nécessairement adaptée à la recommandation d'un exercice d'entraînement.

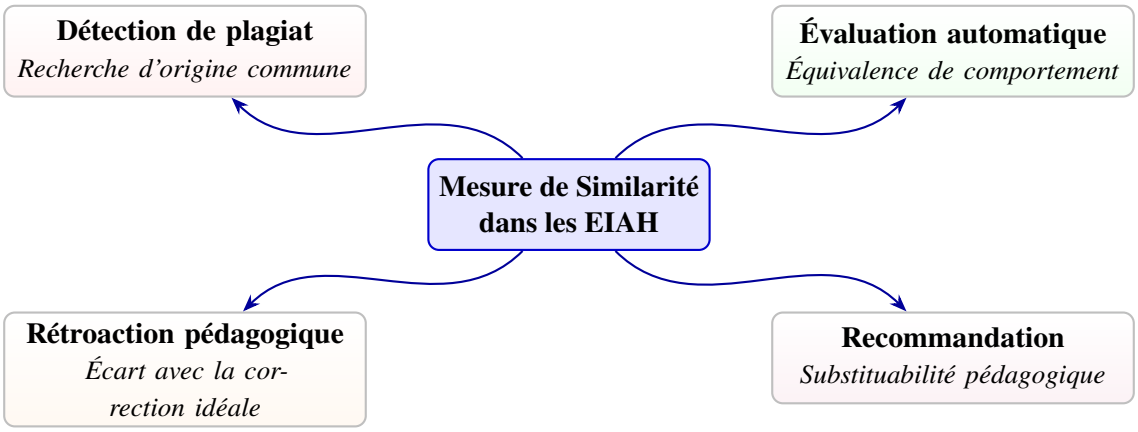


Figure 3.1 – Les quatre grandes finalités de la mesure de similarité dans les EIAH (orientations divergentes à partir d'un même besoin technique).

Dans les premières applications, la comparaison de programmes a surtout été utilisée pour repérer des ressemblances suspectes entre les travaux d'étudiants. Les systèmes de détection de plagiat cherchent alors des indices d'une origine commune : séquences d'instructions, choix de structures, erreurs identiques ou organisation inhabituelle du programme. Des outils comme JPlag comparent les structures des programmes tout en tolérant certaines transformations superficielles, par exemple le renommage des variables ou la modification de la mise en forme [23].

La similarité est également mobilisée pour l'évaluation automatique des réponses. Une solution soumise peut être comparée à une correction de référence ou à un ensemble de solutions déjà validées. Les approches les plus simples examinent la sortie du programme sur plusieurs jeux de tests, tandis que d'autres analysent sa structure, ses dépendances ou ses propriétés sémantiques. La vérification formelle peut aller jusqu'à établir une équivalence comportementale entre deux programmes [18].

Un autre domaine d'application est la recommandation de ressources. Dans une plateforme d'exercices, un enseignant peut souhaiter retrouver rapidement des activités proches d'un exercice existant, soit pour proposer une variante, soit pour construire une évaluation équilibrée. De la même manière, un étudiant peut recevoir un exercice supplémentaire qui reprend une notion déjà

travaillée, sans reproduire exactement le même énoncé. Les travaux sur la recommandation dans les environnements éducatifs cherchent notamment à exploiter les caractéristiques des ressources et les traces d'apprentissage afin d'adapter les propositions au profil de l'apprenant [14, 27].

La comparaison de code intervient également dans la production de rétroactions destinées aux apprenants. Dans l'environnement AlgoPython, Jolivet *et al.* [13] étudient la modélisation et la décision de rétroactions épistémiques adaptées aux erreurs observées et au profil de l'apprenant. Leur approche compare notamment l'arbre syntaxique abstrait d'une production de l'étudiant avec celui d'une solution de référence afin d'identifier les différences structurelles et de les exploiter pour construire un retour pédagogique. La similarité entre ces deux représentations ne sert donc pas ici à détecter une copie ou à recommander un exercice, mais à caractériser l'écart entre la production et la solution attendue. Ce travail montre que les représentations syntaxiques peuvent être utilisées dans plusieurs fonctions d'un EIAH, notamment pour assister l'apprentissage de la programmation.

Deux sources d'information peuvent alors être distinguées :

— les informations externes au programme, comme le texte de l'énoncé, les notions pédagogiques déclarées, le niveau de difficulté ou les métadonnées de l'exercice ;
— les informations internes à la solution, comme les structures de contrôle, les opérations effectuées, l'organisation des données et la complexité approximative du traitement.

En fonction de la source de données considérée, les travaux recensés mettent en œuvre plusieurs familles d'approches complémentaires. Certaines analysent le texte de l'énoncé, d'autres s'appuient sur les concepts pédagogiques ou les traces d'apprentissage, tandis que les approches centrées sur le code étudient sa syntaxe, ses structures de contrôle, ses dépendances ou son comportement à l'exécution. Les méthodes d'apprentissage automatique, notamment les représentations vectorielles et les approches multimodales, ainsi que les méthodes formelles constituent d'autres familles, respectivement fondées sur l'apprentissage de représentations et sur la preuve de propriétés du programme. Enfin, certaines méthodes réutilisent ces représentations pour analyser l'écart entre une production et une référence et produire une rétroaction pédagogique. Aucune de ces approches n'est fiable a 100% : chacune observe une dimension particulière de l'exercice. Nous les présentons donc ici dans leurs grandes lignes afin de situer les méthodes étudiées dans la littérature. Les outils et les étapes de calcul propres à chaque famille seront détaillés dans la section suivante, avant que leurs limites ne soient discutées au regard de l'objectif de ce rapport.

3.3 Quels sont les outils et méthodes utilisés par ces approches ?

Les travaux présentés précédemment ne comparent pas tous les exercices à la même échelle. Selon l'objectif poursuivi, la comparaison peut porter sur le texte de l'énoncé, les concepts pédagogiques associés, les traces d'activité, la forme du programme, ses dépendances ou encore son comportement à l'exécution. Les outils employés sont donc différents, mais ils suivent généralement une chaîne commune modélisée par la figure 3.2.

Cette chaîne répond à quatre questions : quelles données sont disponibles, sous quelle représentation sont-elles transformées, comment la proximité est-elle calculée et à quelle fonction de l'EIAH le résultat est-il destiné ? Cette distinction est importante pour comparer les méthodes. Deux approches peuvent utiliser une même mesure, par exemple la similarité cosinus, tout en comparant des objets différents : deux énoncés, deux corrections ou un énoncé et une correction. À l'inverse, deux approches peuvent partir du même code, mais extraire des informations différentes : l'une peut retenir la structure syntaxique, une autre les dépendances entre instructions et une troisième les propriétés observées à l'exécution. La suite de cette section présente ces familles dans cet ordre, avant d'en discuter les limites.

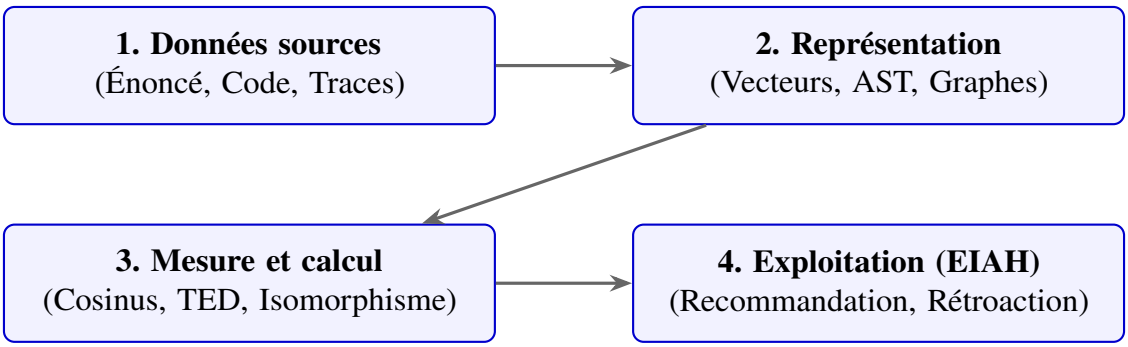


Figure 3.2 – Étapes communes aux méthodes de comparaison d'exercices pédagogiques

Représentation textuelle et traitement automatique du langage. La première famille que on va presenté ici , traite l'énoncé comme un **document textuel**. Une étape de prétraitement consiste à nettoyer le texte, à le segmenter en mots ou en phrases(c'est ce que on appelle tokenisation ou Découpage en lexèmes,lorsque on le segmente en unité de mots ou moins qu'un mots selon la maniere de tokeniser car il existe plusieurs forme de tokenisation) et, selon la méthode, à supprimer les éléments considérés comme peu informatifs(ca c'est communément apppélé Stop word Removal). Les approches lexicales peuvent ensuite représenter chaque énoncé par un vecteur de fréquences , par exemple avec la pondération TF–IDF(au final chaque texte est un vecteur qui a une orientation dans l'espace et deux vecteurs proches dans cet espace seron relativement proches et deux éloigné seront tres séparé ou distant dans l'espace). La proximité entre deux exercices est alors calculée à l'aide d'unedistance ou d'une similarité entre vecteurs , le plus souvent la similarité cosinus.

Les méthodes fondées sur les représentations distribuées vont au-delà du simple partage de mots. Un modèle de type Word2Vec ou FastText transforme les mots en vecteurs appris à partir de leur contexte. Pour l'appariement d'exercices avec des compétences, Tian *et al.* combinent ainsi des représentations lexicales produites par FastText avec une extraction de concepts clés, puis utilisent un classifieur pour estimer les *knowledge components* associés à un exercice [27]. Cette combinaison permet de rapprocher le vocabulaire employé dans l'énoncé de concepts pédagogiques explicites.

Les modèles Transformers proposent une représentation contextualisée des mots et des phrases. BERT et Sentence-BERT produisent des vecteurs qui intègrent le contexte d'apparition des termes ;

Sentence-BERT est notamment conçu pour comparer efficacement des phrases ou des paragraphes [25]. Dans un système de recherche d'exercices, chaque énoncé peut ainsi être transformé en un vecteur, puis comparé à ceux des autres énoncés. Les résultats peuvent servir à la recherche de ressources voisines ou être combinés avec des informations pédagogiques et des traces d'apprentissage [14, 22].

Concepts pédagogiques, annotations et traces d'apprentissage. Une deuxième famille ne cherche pas d'abord une ressemblance de surface : elle représente l'exercice par les compétences ou les notions nécessaires à sa résolution. Ces notions peuvent être définies par des experts sous la forme de *knowledge components*, de concepts algorithmiques ou de compétences du langage. L'outil de comparaison devient alors un système d'annotation ou de classification. Il reçoit le texte de l'exercice, ses métadonnées ou sa correction, et prédit un ensemble de concepts associés. Les scores de proximité sont ensuite calculés à partir de l'intersection, de la pondération ou de la distance entre ces ensembles de concepts [27, 11].

Les systèmes de recommandation peuvent compléter ces informations par les traces d'utilisation de la plateforme : nombre de tentatives, temps passé, taux de réussite, ordre des exercices ou types d'erreurs rencontrées. Des méthodes de filtrage ou de classement exploitent ces données pour proposer un exercice similaire à un apprenant ou pour retrouver, dans une banque, des activités présentant des caractéristiques proches [14]. La similarité n'est alors plus seulement une propriété du contenu : elle dépend aussi des parcours et des comportements observés.

Tests dynamiques et métriques de programme. Dans l'évaluation des exercices de programmation, une autre méthode consiste à exécuter les solutions. Le programme soumis et la correction de référence sont lancés sur un ensemble de tests unitaires ou de tests de propriétés. Les sorties obtenues, les exceptions et les temps d'exécution sont comparés. Cette approche fournit une information comportementale directe et constitue la base de nombreux correcteurs automatiques. La revue systématique de Messer *et al.* montre que les outils d'évaluation combinent souvent tests dynamiques, analyse statique et, dans certains cas, apprentissage automatique pour produire une note ou un retour à l'étudiant [16].

Les outils peuvent aussi extraire des métriques statiques : nombre de lignes, nombre de fonctions, profondeur d'imbrication, complexité cyclomatique, nombre de variables ou fréquence de certaines constructions. Ces valeurs forment un vecteur numérique qui peut être comparé par une distance euclidienne, une distance normalisée ou un modèle de régression. Dans les approches hybrides, les métriques statiques sont associées aux résultats des tests afin de tenir compte à la fois du comportement observé et de l'organisation du code [16, 28].

Arbres syntaxiques, distances d'édition et rétroactions. Les approches syntaxiques commencent par l'analyse du code source à l'aide d'un parseur. Celui-ci vérifie la grammaire du langage et construit un arbre de syntaxe abstraite, ou AST. Chaque nœud représente une construction du programme, par exemple une affectation, une condition, une boucle, un appel de fonction ou une opération. L'AST supprime une partie des détails de mise en forme tout en conservant l'organisation syntaxique du programme.

Les AST peuvent être comparés directement. Les méthodes de détection de clones de Baxter *et al.* utilisent cette représentation pour repérer des fragments identiques ou presque identiques dans des programmes [3]. Pour comparer deux arbres de tailles différentes, la distance d'édition entre arbres compte les opérations nécessaires pour transformer le premier arbre en second : insertion, suppression et substitution de nœuds. L'algorithme de Zhang et Shasha fournit une méthode classique pour calculer cette distance sur des arbres ordonnés [31].

Dans une application à la similarité de code, la distance brute est souvent normalisée par la taille des arbres afin d'obtenir un score comparable entre des programmes courts et longs. Une phase de normalisation peut également remplacer les noms de variables, les valeurs littérales ou certaines variantes syntaxiques par des catégories communes. L'objectif est de comparer l'ossature de la solution plutôt que ses détails accidentels. Les travaux consacrés à l'évaluation de la similarité par AST étudient ainsi la normalisation de l'arbre et l'impact de la distance d'édition sur le score final [26].

La même représentation peut également servir à produire une rétroaction pédagogique. Dans AlgoPython, Jolivet *et al.* comparent l'AST d'une production étudiante avec celui d'une solution de référence afin de localiser les différences et de décider d'une rétroaction épistémique adaptée au profil de l'apprenant [13]. L'AST joue alors le rôle d'une représentation intermédiaire : les différences entre les arbres sont transformées en informations compréhensibles et exploitables dans le parcours d'apprentissage. La similarité sert donc à caractériser l'écart entre une production et une référence, et pas uniquement à classer des exercices.

Graphes de flot, dépendances et graphes sémantiques. L'AST décrit principalement la construction syntaxique du code. Pour représenter plus explicitement son fonctionnement, les chercheurs utilisent différents graphes complémentaires. Un graphe de flot de contrôle (*Control Flow Graph*, CFG) représente les chemins possibles d'exécution : ses nœuds correspondent à des instructions ou à des blocs, et ses arêtes indiquent les transitions possibles entre ces éléments.

Les relations entre les instructions peuvent également être représentées par des graphes de dépendances. Un graphe de dépendances de données (*Data Dependence Graph*, DDG) relie une instruction qui définit une valeur aux instructions qui utilisent cette valeur. Il permet ainsi de représenter les dépendances entre variables, affectations et expressions. Un graphe de dépendances de contrôle décrit quant à lui l'influence d'une condition ou d'une structure de contrôle sur l'exécution d'une instruction. Le graphe de dépendances de programme (*Program Dependence Graph*, PDG) combine généralement ces deux informations : il regroupe les dépendances de données et les dépendances de contrôle au niveau d'une fonction ou d'un programme [30, 24].

À une échelle plus large, le graphe de dépendances système (*System Dependence Graph*, SDG) étend le PDG à plusieurs fonctions en représentant notamment les relations entre les appels de fonctions, leurs paramètres et leurs valeurs de retour. D'autres représentations associent plusieurs types d'informations dans un même modèle. C'est notamment le cas du graphe de propriétés du code (*Code Property Graph*, CPG), qui combine généralement l'AST, le CFG et les dépendances de données et de contrôle. Des outils tels que Joern exploitent ce type de représentation pour effectuer des analyses statiques et rechercher des propriétés structurelles ou sémantiques du code.

Les graphes peuvent être comparés par leurs nœuds, leurs arêtes, leurs chemins ou leurs sous-structures. On peut calculer une distance d'édition de graphes, rechercher des sous-graphes communs ou extraire des caractéristiques structurelles avant d'appliquer une mesure vectorielle. Vujošević-Janičić *et al.* combinent ainsi vérification logicielle et analyse de graphes pour l'évaluation automatique de travaux étudiants [28]. Xiang *et al.* utilisent également des représentations fondées sur les dépendances du code pour comparer une solution étudiante à une solution de référence dans le cadre d'exercices de programmation [30].

La sélection du graphe dépend du compromis recherché entre richesse de la représentation, précision de l'analyse et coût de calcul. L'AST est relativement compact et facile à extraire. Le CFG décrit l'organisation du flot d'exécution, tandis que le DDG et le PDG rendent explicites les relations entre les calculs et les variables. Le SDG permet d'étudier les interactions entre fonctions, alors qu'un CPG rassemble plusieurs de ces dimensions dans une représentation unifiée. L'étude comparative de Paiva *et*

al. examine plusieurs représentations de graphes sémantiques et compare notamment leur coût et leur pouvoir descriptif [21]. Les travaux de synthèse sur la similarité sémantique des exercices regroupent ces méthodes avec les approches fondées sur les graphes de contrôle et de dépendances [24].

Ces représentations peuvent également être utilisées pour produire une rétroaction à partir d'une analyse statique du code. La comparaison des PDG, des DDG ou des CPG permet par exemple de localiser une dépendance manquante, une structure de contrôle inattendue ou une utilisation incorrecte d'une variable. Paiva *et al.* comparent plusieurs représentations graphiques afin d'évaluer leur richesse, leur coût et leur capacité à supporter une rétroaction automatique [21]. Les graphes ne servent donc pas uniquement à calculer un score de similarité : ils peuvent aussi rendre les différences entre une solution étudiante et une référence plus explicables.

Représentations vectorielles et modèles appris. Les méthodes d'apprentissage automatique transforment le texte ou le code en vecteurs denses, appelés *embeddings*. Une première stratégie consiste à extraire des chemins dans l'AST et à apprendre leur représentation. Le modèle code2vec, par exemple, agrège des chemins AST pour produire un vecteur de code [2] ; code2seq exploite également les chemins de l'arbre dans un modèle séquence-à-séquence [1].

Les modèles pré-entraînés spécialisés dans le code constituent une autre famille. CodeBERT apprend conjointement à partir de code et de langage naturel, ce qui permet de rapprocher une implémentation de sa documentation ou de comparer des fragments dans un espace commun [7]. À la différence d'un modèle qui ne considère que la séquence de tokens, GraphCodeBERT introduit le flot de données entre variables et des objectifs d'apprentissage sensibles à cette structure. Il utilise notamment une attention guidée par le graphe pour intégrer les relations de données dans la représentation [9].

Évaluation vectorielle par CodeBERT : spécialisation et limites. Face aux limites des règles explicites fondées sur les tags ou les AST, nous avons également envisagé l'utilisation d'un modèle d'apprentissage profond spécialisé dans le code : CodeBERT. Celui-ci produit des plongements vectoriels du code, appris à partir de tâches de compréhension et de génération de programmes. Son évaluation s'appuie notamment sur plusieurs objectifs associés au génie logiciel :

- la **détection de clones**, qui consiste à déterminer si deux fragments de code sont des clones selon un apprentissage binaire ;
- le **code search**, qui vise à retrouver un fragment de code à partir d'une intention exprimée en langage naturel ;
- le **refinement**, qui cherche à améliorer la qualité ou l'efficacité d'un code sans en modifier la fonction, par exemple en transformant une boucle naïve en compréhension de liste ;
- la **translation**, qui consiste à convertir automatiquement un programme d'un langage vers un autre, par exemple de Python vers Java, au moyen d'une architecture séquence–séquence.

Ces tâches rendent les représentations de CodeBERT particulièrement pertinentes pour la recherche de similarité fonctionnelle entre programmes. Elles ne suffisent toutefois pas à mesurer la substituabilité pédagogique. En effet, un modèle ajusté pour distinguer des clones ou pour rapprocher des solutions fonctionnellement similaires tend à produire un signal de proximité binaire ou fortement corrélé à la similarité de résolution. Il ne permet donc pas nécessairement de distinguer un exercice dont la résolution est trop proche, et qui rendrait sa réalisation peu utile à un étudiant ayant déjà réussi le premier, d'un exercice partageant le même thème tout en exigeant un effort cognitif nouveau. Les poids du modèle ne sont pas alignés sur des critères de compétences pédagogiques : un score élevé peut ainsi

traduire une proximité fonctionnelle sans constituer une preuve de substituabilité dans un parcours d'apprentissage.

Pour comparer deux exercices, ces modèles peuvent produire un vecteur pour chaque énoncé, correction ou paire énoncé–code. La proximité est ensuite calculée par similarité cosinus, distance euclidienne ou score appris. Les modèles siamois et l'apprentissage contrastif rapprochent les représentations de deux objets considérés comme similaires et éloignent les autres exemples. Dans le domaine des exercices multimodaux, BCRNet combine texte et images avec une perte contrastive et un mécanisme d'appariement texte–image afin d'apprendre une représentation utilisable pour la recherche et la recommandation [19].

Vérification formelle et comparaison comportementale. Les méthodes formelles ne se limitent pas à mesurer une proximité : elles cherchent à démontrer une propriété du programme. Le code peut être traduit en une représentation logique, puis soumis à un vérificateur ou à un solveur. Pour prouver l'équivalence de deux programmes, on peut utiliser la réécriture des boucles en fonctions récursives, l'induction fonctionnelle et un solveur de satisfaisabilité modulo théories (SMT). Dans ses travaux sur l'équivalence de programmes, Milovančević s'appuie sur le vérificateur Stainless et sur des preuves d'induction fonctionnelle pour comparer des programmes candidats à des programmes de référence [17].

Dans un contexte de cours, la vérification formelle peut compléter les tests et distinguer des solutions qui ont le même comportement sur les entrées testées mais qui reposent sur des structures de résolution différentes. L'expérience de Vafeiadis *et al.* illustre l'utilisation d'un vérificateur d'équivalence conjointement avec une infrastructure de correction par tests [18]. Cette famille produit donc une information très forte sur le programme, mais elle répond à une question de correction ou d'équivalence qui n'est pas exactement celle de la substituabilité pédagogique.

Synthèse des méthodes mobilisées. Les approches recensées peuvent être regroupées selon la source d'information qu'elles exploitent, comme le résume la figure 3.3. Le schéma ne représente pas une hiérarchie de méthodes ni un classement de leur coût : chaque famille observe une dimension différente des exercices. Dans ce travail, les tests unitaires n'ont pas constitué une méthode de similarité étudiée en profondeur. Ils ont seulement été envisagés lors d'une première exploration comme un moyen de vérifier le comportement des programmes. Les résultats présentés ensuite reposent sur l'analyse statique des corrections de référence ; aucune conclusion bibliographique ou expérimentale spécifique sur le coût des tests unitaires n'est donc tirée ici.

Ces familles sont complémentaires. Les méthodes textuelles et conceptuelles décrivent l'intention déclarée de l'exercice ; les traces décrivent son usage ; l'AST décrit sa construction syntaxique ; les graphes décrivent ses flux et ses dépendances ; les modèles vectoriels apprennent des représentations distribuées ; enfin, la vérification formelle cherche à établir une propriété logique ou comportementale. Les outils associés vont de la similarité cosinus et de la distance d'édition aux algorithmes de comparaison de graphes, aux modèles Transformers et aux solveurs formels.

Dans le cadre de ce rapport, aucune de ces représentations ne suffit à elle seule à caractériser la substituabilité pédagogique. Nous retenons donc une analyse centrée sur les corrections de référence, qui combine une mesure globale de caractéristiques, une comparaison d'AST normalisés et une représentation des structures de contrôle. Ce choix conserve l'interprétabilité des méthodes syntaxiques et structurelles tout en permettant de comparer des stratégies de résolution. Les limites de ces outils, ainsi que les raisons pour lesquelles certaines méthodes ne sont pas retenues dans notre pipeline, sont examinées dans la section suivante.

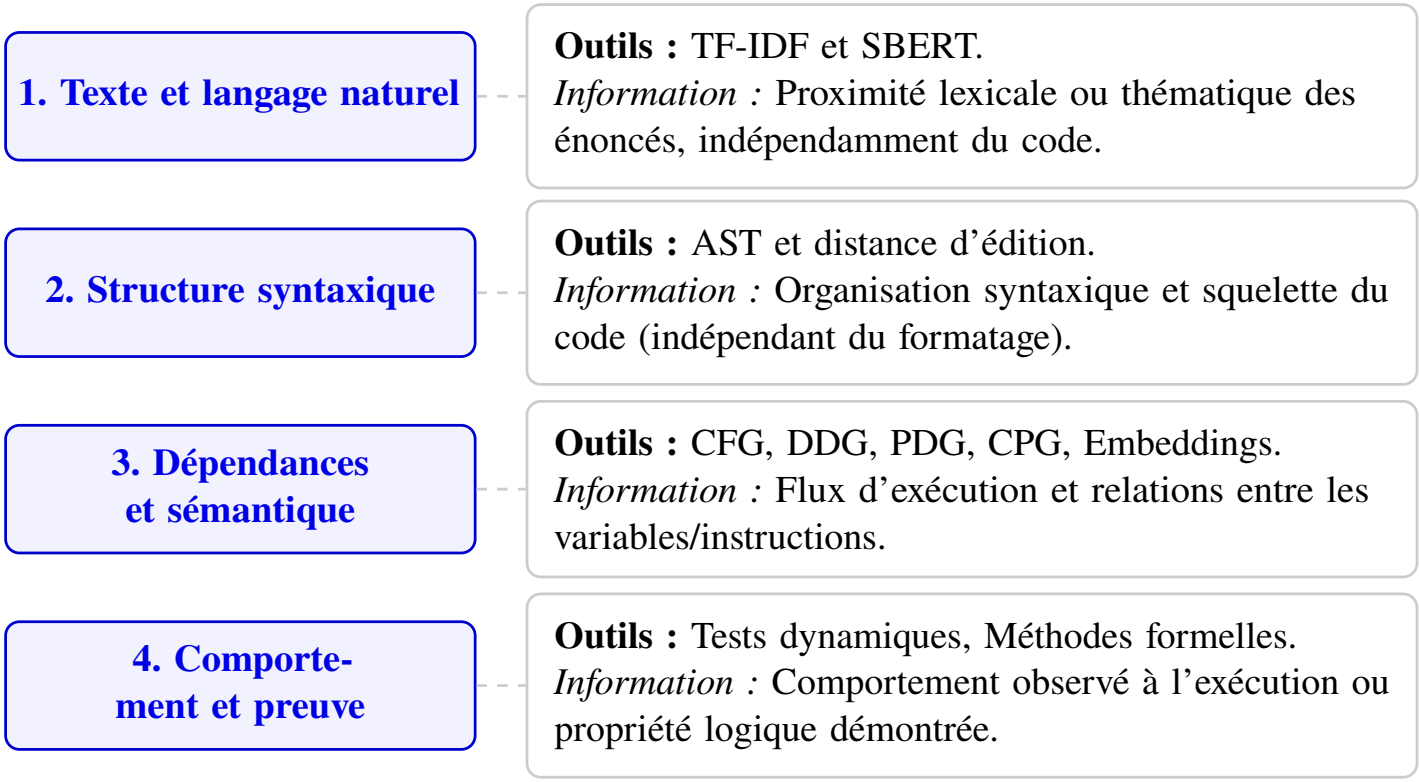


FIGURE 3.3 – Familles de représentations mobilisées pour comparer des exercices de programmation

3.4 Discussion sur les limites des méthodes

Les méthodes présentées dans les sections précédentes permettent de comparer des exercices selon des dimensions différentes, mais aucune ne mesure à elle seule la substituabilité pédagogique. Cette limite tient d'abord à la nature de la notion étudiée. La substituabilité ne dépend pas uniquement de la forme du code : elle suppose également que deux exercices mobilisent des connaissances proches, visent des apprentissages comparables et présentent un niveau de difficulté compatible. Or ces informations ne sont pas toujours explicitement décrites dans les énoncés ou dans les corrections de référence.

Les approches fondées sur le texte constituent un premier moyen de retrouver des exercices traitant de thèmes voisins. Elles sont relativement simples à mettre en œuvre et peuvent être appliquées à un corpus important. Leur principale faiblesse est cependant leur dépendance à la formulation. Deux énoncés qui mobilisent la même démarche algorithmique peuvent employer un vocabulaire très différent. À l'inverse, deux énoncés partageant de nombreux termes peuvent demander des solutions très différentes. Les représentations par TF–IDF restent sensibles au partage lexical, tandis que les embeddings contextuels dépendent du modèle utilisé et de ses données d'entraînement. Elles décrivent donc davantage la proximité thématique ou linguistique que la structure effective de la résolution.

Les annotations pédagogiques et les traces d'apprentissage pourraient apporter une information plus directement liée aux objectifs de formation. Des notions, des compétences, des niveaux de difficulté, le temps de résolution ou le taux de réussite permettraient par exemple de compléter l'analyse du code. Ces données ne sont toutefois pas disponibles dans le contexte étudié. Le corpus ne contient pas d'annotation de substituabilité entre les exercices et aucune trace d'activité d'apprenants n'est exploitable. Cette situation de *cold start* empêche notamment d'entraîner de manière fiable un modèle supervisé ou de vérifier directement qu'un score automatique correspond à un jugement pédagogique.

Les méthodes fondées sur l'exécution fournissent une information utile sur le comportement d'un programme. Elles peuvent vérifier que deux solutions produisent les mêmes résultats sur un ensemble d'entrées et mesurer certains aspects de leur performance. Elles ne suffisent cependant pas à caractériser la stratégie algorithmique. Deux programmes peuvent obtenir les mêmes sorties sur les tests disponibles tout en reposant sur des structures très différentes. De plus, une égalité de comportement ne signifie pas nécessairement que les deux exercices poursuivent le même objectif pédagogique. Les tests ne couvrent par ailleurs qu'une partie des comportements possibles et peuvent manquer des cas limites.

Les AST et les distances d'édition offrent une représentation plus stable que le texte brut, car ils réduisent l'influence de la mise en forme et de certains détails lexicaux. Ils restent néanmoins sensibles aux choix d'implémentation. Un changement de boucle, l'extraction d'une fonction ou l'introduction d'une variable intermédiaire peut produire une distance importante alors que la démarche générale reste proche. À l'inverse, deux arbres peuvent présenter une organisation similaire tout en correspondant à des traitements différents. La normalisation atténue ces effets, mais chaque règle de normalisation introduit un choix qui peut supprimer une information pertinente ou rapprocher à tort des programmes distincts.

Les graphes de flot et de dépendances décrivent plus explicitement le fonctionnement du programme. Les CFG, DDG, PDG et CPG peuvent notamment représenter les chemins d'exécution, les dépendances entre variables et les relations entre instructions. Cette richesse permet de dépasser certaines limites des AST, mais elle augmente le coût d'extraction et de comparaison. La construction de ces graphes peut aussi dépendre du langage, de la qualité de l'analyse statique et de la capacité des outils à traiter les appels de fonctions, les bibliothèques ou les effets indirects. Une représentation plus riche n'est donc pas automatiquement plus pertinente pour une comparaison pédagogique.

Les modèles appris et les représentations vectorielles peuvent capturer des relations difficiles à

définir manuellement et s'adapter à de grands corpus. Leur interprétation est toutefois moins directe : lorsqu'une paire obtient un score élevé, il est difficile d'identifier précisément les caractéristiques responsables de cette proximité. Leur comportement dépend également du corpus d'entraînement, du langage de programmation et du domaine représenté. Dans le présent travail, l'absence de paires annotées empêche en outre d'apprendre une fonction spécifiquement orientée vers la substituabilité.

Enfin, les méthodes formelles apportent des garanties fortes lorsqu'il est possible de prouver une propriété du programme. Leur coût de mise en œuvre et leurs conditions d'application limitent cependant leur utilisation sur un corpus hétérogène d'exercices. Surtout, l'équivalence comportementale répond à une question différente de celle étudiée ici : deux exercices peuvent demander des résultats différents tout en sollicitant une démarche algorithmique comparable.

Ces limites conduisent à privilégier une approche complémentaire plutôt qu'un score unique présenté comme une vérité pédagogique. Dans ce rapport, les corrections de référence constituent la source principale d'information, car elles permettent d'observer directement les choix structurels de résolution. Les approches retenues combinent une mesure globale de caractéristiques, une comparaison d'AST normalisés et une représentation des structures de contrôle. Ce choix vise un compromis entre robustesse aux variations de code, interprétabilité des résultats et coût de calcul. Les scores obtenus doivent être compris comme des indicateurs destinés à filtrer des paires candidates . Une validation parun enseignant reste nécessaire pour vérifier la proximité des notions visées,de la difficulté et des objectifs d'apprentissage.

Afin d'illustrer la nécessité de croiser les approches, la figure 3.4 résume « l'angle mort » inhérent à chaque grande famille méthodologique vis-à-vis de la notion de substituabilité.

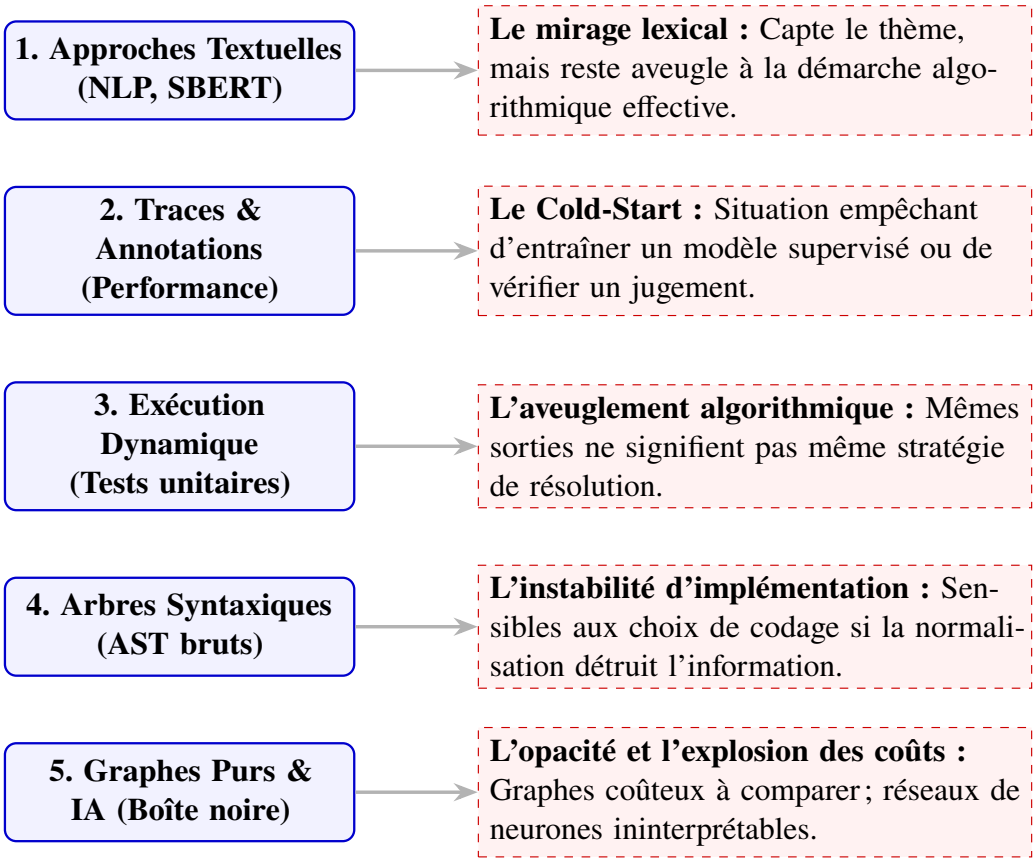


Figure 3.4 – Cartographie des limites : pourquoi chaque approche individuelle échoue à caractériser seule la substituabilité pédagogique.

3.5 Pistes explorées et écartées

La définition du pipeline retenu est le résultat de plusieurs explorations préliminaires. L'objectif de ces essais n'était pas seulement d'obtenir le meilleur score possible, mais aussi d'identifier des représentations compatibles avec les données disponibles, suffisamment interprétables et réutilisables sur un corpus d'exercices potentiellement hétérogène. Plusieurs pistes ont ainsi été étudiées sans être retenues comme méthodes principales.

Étiquetage par mots-clés et modèles génératifs. Une première piste, explorée dans les travaux préliminaires d'Adel ZIANI, consistait à caractériser chaque exercice au moyen d'étiquettes conceptuelles telles que *boucle*, *récursivité*, *liste* ou *condition*. L'extraction de ces étiquettes a notamment été testée à l'aide de modèles de langage génératifs (LLM), afin d'identifier automatiquement les notions mobilisées par les corrections [32].

Cette approche s'est toutefois révélée difficile à stabiliser. Les réponses des LLM peuvent varier selon la formulation de la requête et le contexte fourni ; les étiquettes produites et les regroupements obtenus ne sont donc pas nécessairement reproductibles ni cohérents d'une exécution à l'autre. Par ailleurs, une liste fermée de mots-clés impose un vocabulaire défini à l'avance. Si une correction mobilise une structure ou une notion absente de cette liste, cette information n'est pas représentée. Dans une représentation vectorielle fondée uniquement sur ces étiquettes, l'exercice peut alors présenter un vecteur nul ou incomplet, ce qui fausse le calcul de similarité. Les LLM ont donc été conservés comme piste exploratoire, mais ils ne constituent pas la base du pipeline retenu.

Extraction de motifs syntaxiques avec Tree-sitter. Au cours du présent stage, une autre piste a consisté à remplacer cet étiquetage génératif par une extraction déterministe de motifs syntaxiques. L'outil Tree-sitter a été utilisé pour analyser les corrections et rechercher explicitement des structures telles que les conditions, les boucles, les appels de fonctions ou les retours. L'objectif n'était plus de comparer directement les arbres entiers, mais de produire automatiquement des caractéristiques structurelles à partir du squelette du code.

Cette approche permet de rendre l'extraction reproductible et indépendante des variations de réponse d'un modèle génératif. Elle reste cependant limitée par les motifs définis dans le programme d'analyse. Le parseur ne peut identifier que les structures qui ont été anticipées et codées dans les règles d'extraction ; une démarche algorithmique non prévue est ignorée ou réduite à une représentation incomplète. Elle reproduit ainsi, sous une forme plus déterministe, le biais d'une liste fermée de tags.

Comparaison des énoncés par TF–IDF et SBERT. Une première piste consistait à comparer directement les textes des énoncés. Cette approche présente l'avantage de partir d'une information immédiatement disponible, sans nécessiter l'analyse du code source. La représentation TF–IDF fournit ainsi un point de référence simple et rapide. Pour chaque énoncé, elle attribue à chaque terme un poids qui dépend de deux éléments : la fréquence du terme dans l'énoncé et sa rareté dans l'ensemble du corpus. Un mot très fréquent dans un document mais peu présent dans les autres énoncés reçoit donc un poids important, tandis qu'un mot présent partout contribue peu à différencier les exercices. Chaque énoncé est ainsi transformé en un vecteur numérique, puis les vecteurs de deux exercices sont comparés à l'aide de la similarité cosinus. Cette mesure évalue si les deux vecteurs pointent dans des directions proches, ce qui revient principalement à mesurer le partage de termes importants.

Des représentations produites par SBERT ont également été envisagées afin de dépasser cette comparaison fondée sur les mots. Le modèle transforme chaque phrase, ou chaque énoncé, en un

vecteur dense appris à partir du contexte des mots. Deux formulations différentes peuvent ainsi être rapprochées lorsqu'elles expriment une idée similaire, même si elles ne partagent pas exactement le même vocabulaire. La similarité entre les vecteurs SBERT est elle aussi généralement calculée avec la similarité cosinus, mais le score porte alors sur une représentation sémantique et non plus seulement sur la fréquence des termes.

Ces deux méthodes ne décrivent cependant pas directement la manière dont le programme doit être construit. TF–IDF reste sensible aux mots employés dans l'énoncé, tandis que SBERT peut rapprocher des phrases dont le sens général est semblable sans distinguer les structures de contrôle ou les algorithmes attendus. Elles ont donc été utilisées comme premières approximations de la proximité thématique.

Ces essais ont fourni un signal de proximité thématique, mais ce signal ne permet pas de caractériser suffisamment la démarche de résolution. Des énoncés peuvent employer des termes différents tout en demandant des algorithmes proches. Inversement, deux énoncés peuvent partager un vocabulaire lié aux listes, aux boucles ou aux fonctions tout en nécessitant des solutions structurellement différentes. Ces méthodes ont donc été conservées comme éléments exploratoires ou comme informations complémentaires possibles, mais elles n'ont pas été retenues comme source principale de similarité.

La dynamique entre ces deux méthodes est illustrée par la figure 3.5, qui compare directement les scores de similarité TF–IDF et SBERT pour l'ensemble des paires d'énoncés du corpus.

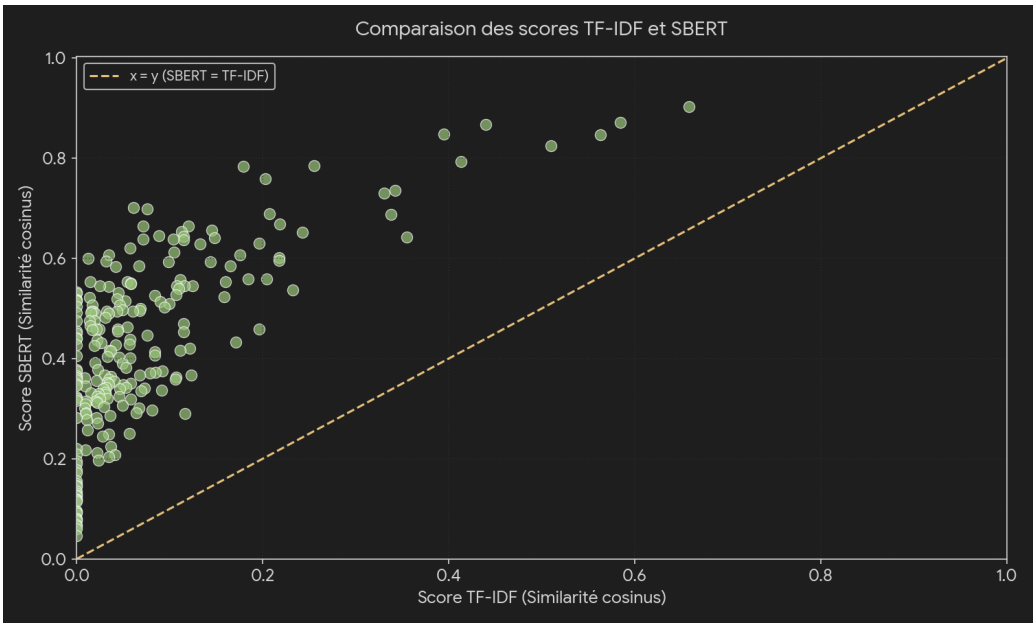


Figure 3.5 – Comparaison des scores de similarité cosinus TF–IDF et SBERT pour l'ensemble des paires d'énoncés. La ligne pointillée représente l'égalité des scores.

L'analyse de cette distribution confirme empiriquement les limites de ces approches textuelles. La quasi-totalité des points se situe au-dessus de la ligne d'égalité, démontrant que SBERT attribue des scores systématiquement plus élevés (moyenne de 0,42) que TF–IDF (moyenne de 0,07). L'importante dispersion verticale sur la gauche du graphique montre la capacité de SBERT à rapprocher des énoncés partageant une sémantique commune mais aucun vocabulaire (TF–IDF proche de 0). Cependant, l'absence de regroupements (clusters) distincts prouve que ces scores thématiques peinent à segmenter finement les exercices selon leur démarche de résolution algorithmique.

Pour illustrer concrètement ces limites, nous pouvons examiner un cas représentatif tiré de notre corpus, mettant en évidence les « illusions » lexicales :

Exemple : l'illusion lexicale (TF–IDF élevé, structures différentes).

— *Énoncé A :* « Écrire un programme qui demande une somme cible entre 2 et 12, puis affiche toutes les combinaisons possibles de deux dés à six faces donnant cette somme. Afficher aussi le nombre total de combinaisons. »

— *Énoncé B :* « Écrire un programme qui demande à l'utilisateur un nombre de dés (entre 1 et 8) et une somme cible. Le programme doit afficher combien de combinaisons de résultats permettent d'obtenir cette somme avec des dés à six faces. »

Pour cette paire, les deux méthodes accordent des scores très élevés (TF–IDF = 0,65 ; SBERT = 0,90) en raison du fort champ lexical partagé (dés, somme, combinaisons). Pourtant, le premier énoncé se résout par de simples boucles imbriquées fixes, tandis que le second requiert une approche récursive ou une gestion dynamique pour un nombre variable de dés. Les métriques textuelles échouent à capturer cette rupture de complexité structurelle.

Comparaison de séquences de jetons (stratégie de JPlag). JPlag compare les programmes en deux étapes [23]. Chaque programme est d'abord transformé en une suite de jetons : les noms de variables et les valeurs sont effacés, seuls restent des éléments comme « début de boucle » ou « affectation ». Les deux suites sont ensuite comparées en cherchant les plus longs morceaux identiques qu'elles ont en commun (*Greedy String Tiling*).

La première étape rejoint notre démarche : la normalisation de l'AST efface elle aussi les noms et les valeurs. La seconde ne correspond pas à notre objectif. JPlag cherche de longs passages identiques, signe qu'un programme a été recopié. Or deux exercices substituables ne sont pas des copies : ils suivent la même démarche avec une écriture qui peut varier. Remplacer une boucle `for` par une boucle `while` ou changer l'ordre de deux instructions suffit à couper la suite de jetons en petits morceaux, alors que la démarche est inchangée. De plus, une suite de jetons est « à plat » : elle ne dit pas quelle instruction se trouve à l'intérieur de quelle boucle, alors que cette imbrication décrit justement la démarche de résolution. Nous avons donc gardé l'idée de normalisation, mais remplacé la comparaison de séquences par une comparaison d'arbres.

Graphes de dépendances et modélisations hybrides (CFG, PDG, CPG, CSSG). Pour dépasser la rigidité topologique des Arbres Syntaxiques Abstraits (AST), la modélisation des flux d'exécution a été explorée afin de capturer l'essence sémantique du code. Contrairement à un AST qui décrit la forme syntaxique stricte, les graphes de flot de contrôle (CFG) modélisent les différents chemins d'exécution possibles, tandis que les graphes de dépendances de données (DDG) et de programme (PDG) explicitent les relations entre la définition d'une variable et ses utilisations ultérieures. Ces graphes présentent l'avantage majeur de rester stables même si un étudiant réordonne des instructions indépendantes ou modifie la syntaxe de surface.

Pour obtenir une représentation absolue, des approches hybrides telles que les CPG (Code Property Graphs) ou les CSSG (Code Syntax and Semantics Graphs) ont été étudiées. Le modèle CSSG, par exemple, fusionne l'AST, le CFG et le PDG en une structure de données unique, capturant simultanément la syntaxe et la sémantique d'exécution.

Cependant, cette piste s'est heurtée à un obstacle technique majeur lors de la phase de comparaison. Évaluer la similarité exacte entre deux graphes aussi riches et denses nécessite de calculer des distances d'édition de graphes (Graph Edit Distance) ou de rechercher des isomorphismes de sous-graphes. Ces opérations mathématiques étant d'une complexité algorithmique NP-difficile, le temps de calcul explose de manière exponentielle. Cette approche s'est donc révélée inapplicable pour évaluer et comparer à

grande échelle un corpus important d'exercices en temps raisonnable. Elle demeure une perspective théorique d'enrichissement, mais a été écartée du pipeline principal.

Modèles appris et embeddings de code (CodeBERT et GraphCodeBERT). Face aux limites combinatoires des graphes, l'utilisation de modèles de langage pré-entraînés spécialisés dans le code source a été envisagée pour projeter les programmes dans un espace vectoriel dense (embeddings). Le modèle CodeBERT s'appuie sur une architecture Transformer entraînée conjointement sur du langage naturel et du code, tandis que son évolution, GraphCodeBERT, y intègre la structure du flux de données (data flow) pour mieux saisir la logique algorithmique sous-jacente.

Si ces modèles excellent dans la représentation globale du code, ils présentent une limite conceptuelle fondamentale par rapport à notre objectif. Ces architectures sont pré-entraînées sur des tâches de génie logiciel rudimentaires, telles que la recherche de code (Code Search) ou la détection binaire de clones (déterminer si un code A est le plagiat d'un code B). Par conséquent, les espaces vectoriels qu'ils génèrent sont optimisés pour la similarité fonctionnelle stricte, et échouent à mesurer la notion de substituabilité pédagogique.

Ces modèles ne parviennent pas à différencier un exercice dont la démarche de résolution est sémantiquement identique (et donc inutile à recommander à un étudiant qui maîtrise déjà le concept) d'un exercice partageant une thématique proche mais exigeant une structure de contrôle inédite (et donc pertinent pour la progression). Pour devenir exploitables dans un Environnement Informatique d'Apprentissage Humain (EIAH), ces modèles nécessiteraient un réentraînement profond (fine-tuning) sur de larges volumes de paires d'exercices annotées par des enseignants selon leur valeur d'apprentissage, une ressource actuellement indisponible à l'échelle requise. Ils ont donc été écartés au profit de représentations plus interprétables et ne nécessitant pas d'apprentissage préalable.

Apprentissage contrastif pour la similarité pédagogique. Pour lever l'obstacle des paires annotées, une méthode d'apprentissage contrastif a été proposée. Elle s'inspire de BCRNet [19] et des travaux sur l'apprentissage contrastif d'images, de phrases et de code [5, 8, 12]. L'objectif est d'apprendre une représentation vectorielle unique de chaque exercice, qui réunit l'énoncé et la correction, sans recourir à des paires étiquetées manuellement.

Pour chaque exercice, on génère deux *vues* qui conservent le même contenu pédagogique et le même comportement du code, mais dont la forme diffère. Ces deux vues forment une **paire positive** que le modèle apprend à rapprocher dans l'espace de représentation. Les vues des autres exercices du lot d'entraînement forment des **paires négatives** que le modèle apprend à éloigner, au moyen de la perte InfoNCE [20]. Le signal d'apprentissage vient donc uniquement de la structure du problème : deux vues d'un même exercice doivent être proches. La méthode reposait sur trois éléments :

- **les vues de l'énoncé**, obtenues par *dropout* appliqué à l'encodeur de texte, à la manière de SimCSE [8]. Le masquage de mots utilisé par BCRNet a été écarté, car il fournit un signal faible à un modèle déjà pré-entraîné par masquage (*masked language modeling*) ;
- **les vues de la correction**, obtenues par des transformations qui préservent la sémantique et que la littérature a déjà validées [12, 4] : renommage des variables, suppression des commentaires, reformatage et réordonnancement d'instructions indépendantes ;
- **une fusion multimodale**, sur le modèle de BCRNet : l'énoncé et le code sont encodés séparément, leurs vecteurs sont concaténés, puis projetés dans un espace latent commun qui fournit un plongement unique par exercice.

Cette piste n'a pas été retenue, car elle repose sur deux types de paires et nous ne pouvions en construire qu'un seul. Les paires positives sont faciles à obtenir : il suffit de créer des variantes bruitées

d'un même exercice. Les paires négatives, en revanche, doivent être de vrais exemples d'exercices *non similaires*. Or, dans notre situation de démarrage à froid, aucune annotation ne permet de savoir quels exercices sont différents. Prendre simplement les autres exercices du lot comme négatifs reviendrait à supposer qu'ils ne se ressemblent pas, alors que c'est justement ce que l'on cherche à mesurer : deux exercices du corpus peuvent très bien être similaires. Sans négatifs fiables, le modèle ne peut pas apprendre correctement la notion de similarité.

L'idée des variantes bruitées a toutefois été conservée sous une autre forme : les variantes contrôlées du chapitre 5 ne servent pas à entraîner un modèle, mais à *vérifier* le comportement des mesures.

Fusion immédiate des signaux. Enfin, une fusion directe de toutes les mesures disponibles aurait pu produire un score unique combinant texte, métriques, AST et structures de contrôle. Cette solution aurait simplifié le classement des paires, mais elle aurait également masqué la contribution de chaque représentation. Elle aurait en outre nécessité de fixer des pondérations sans disposer d'une vérité terrain ou de jugements d'enseignants pour les apprendre.

Pour cette raison, des méthodes sont d'abord étudiées séparément. Leurs résultats pourront ensuite être comparés et combinés de manière explicite, après analyse de leur comportement sur les variantes contrôlées. Ce choix conduit au pipeline présenté dans le chapitre suivant. Celui-ci reposera sur trois représentations complémentaires des corrections : des caractéristiques globales du code, des arbres syntaxiques normalisés et une représentation centrée sur l'organisation des structures de contrôle. Le nom, la construction et le calcul précis de chacune de ces approches seront introduits au début du chapitre suivant, avant la présentation de leurs résultats expérimentaux. —

Chapitre 4

Ma proposition

Ce chapitre présente le pipeline multi-niveaux développé pour évaluer la substituabilité algorithmique entre des exercices de programmation. L'objectif n'est pas de réduire la similarité à une seule propriété du code, mais de comparer les corrections selon trois niveaux d'abstraction complémentaires : la structure globale, la topologie syntaxique détaillée et l'organisation structurelle abstraite du programme. Une interface Streamlit complète le pipeline : elle permet de lancer les comparaisons, de consulter les trois scores et d'inspecter les arbres produits à chaque étape.

4.1 Architecture globale du pipeline

Pour chaque paire de programmes (A, B), le pipeline construit plusieurs représentations du code et calcule trois mesures distinctes :

1. l'**EIIA**, qui compare la difficulté global extraits de l'AST normalisé ;
2. le **TSED**, distance d'édition calculée sur des AST normalisés afin d'évaluer finement la topologie syntaxique ;
3. le **NCST**, qui compare, par un appariement ordonné, les nœuds abstraits d'un arbre structurel normalisé.

Ces trois mesures sont calculées séparément pour chaque paire. Le pipeline produit donc trois scores indépendants, chacun décrivant une dimension particulière de la ressemblance entre les programmes. Cette organisation multi-vues permet notamment de repérer les cas où deux corrections ont une complexité globale proche, mais des organisations structurelles différentes. Elle est cohérente avec les travaux qui distinguent la similarité de contenu, de structure et de finalité pédagogique [22, 10].

La chaîne de traitement est résumée par la figure 4.1. Le code source est d'abord analysé par Tree-sitter, qui produit un arbre syntaxique concret (CST). Cet arbre alimente deux branches : la première le convertit en AST simplifié puis le normalise, ce qui sert de base commune à l'EIIA et au TSED ; la seconde construit directement le NCST brut à partir du CST, le normalise puis apparie les nœuds abstraits. Les trois scores sont exportés dans des colonnes séparées. L'interface Streamlit peut ensuite calculer, à titre facultatif, un score pondéré dont les poids sont choisis par l'utilisateur.

Le tableau 4.1 résume ce que chaque approche conserve, ignore et mesure.

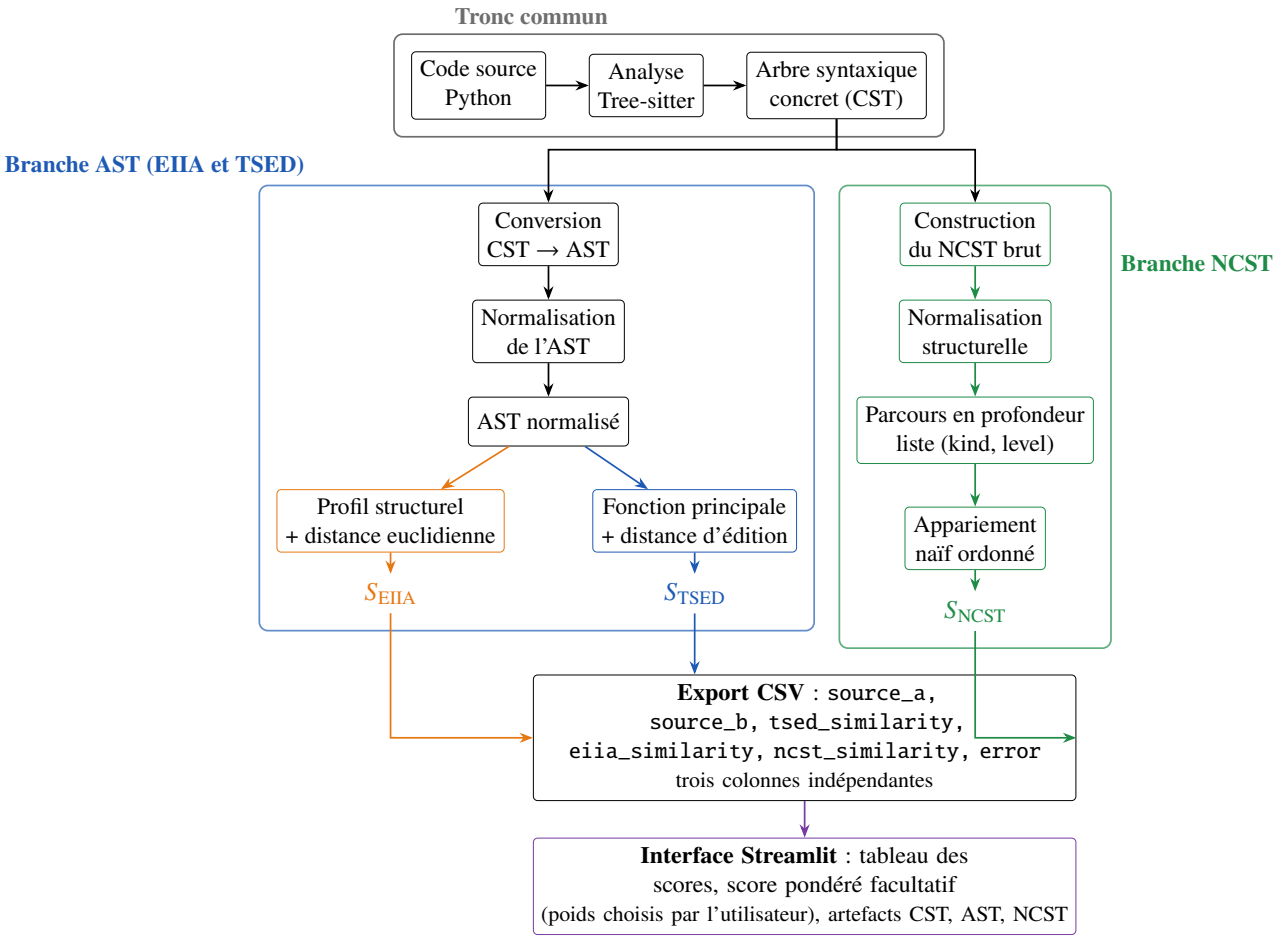


FIGURE 4.1 – Architecture globale du pipeline d'évaluation de la similarité multi-niveaux. L'EIIA et le TSED partagent l'AST normalisé ; le NCST est construit directement à partir du CST. L'interface Streamlit exploite le CSV produit par le pipeline.

	EIIA	TSED	NCST
Représentation	AST normalisé, résumé en vecteur	AST normalisé complet	NCST normalisé, linéarisé en liste (*kind*, *level*)
Information conservée	Effectifs de nœuds et profondeur	Labels AST, opérateurs, ordre et hiérarchie	Catégories structurelles, niveau d'imbrication, ordre du parcours
Information ignorée	Position et agencement des nœuds	Noms et valeurs concrètes	Noms, valeurs, expressions, opérateurs, paramètres, noms de fonctions
Calcul	Distance euclidienne normalisée	Distance d'édition de Zhang-Shasha	Appariement naïf ordonné, indice de Sørensen-Dice
Question posée	Les programmes ont-ils une taille et une complexité comparables ?	Les arbres sont-ils proches nœud à nœud ?	Les programmes partagent-ils les mêmes catégories abstraites aux mêmes niveaux, dans le même ordre ?

TABLE 4.1 – Synthèse des trois niveaux de représentation du pipeline.

4.2 Évaluation de l'information indépendante de l'arbre (EIIA)

L'EIIA fournit une mesure macroscopique de la proximité entre deux programmes. Elle repose sur un profil structurel extrait en parcourant l'ensemble des nœuds de l'AST normalisé. Pour un programme A, le profil est représenté par le vecteur à neuf composantes :

$$V_A = (n_{nodes}, n_{operators}, n_{operands}, n_{assignments}, n_{selections}, n_{iterations}, d_{max}, n_{calls}, n_{returns}). \quad (4.1)$$

Le tableau 4.2 précise les types de nœuds comptés pour chaque composante.

Composante	Signification	Nœuds comptés
`n_nodes`	taille de l'arbre	tous les nœuds
`n_operators`	opérateurs	`BINARY_OP, COMPARE, BOOLEAN_OP, UNARY_OP`
`n_operands`	opérandes	`IDENTIFIER, NUMBER, STRING, BOOLEAN, NULL`
`n_assignments`	affectations	`ASSIGN, AUG_ASSIGN`
`n_selections`	sélections	`IF, ELIF, ELSE, TERNARY`
`n_iterations`	répétitions	`FOR, WHILE`
`d_max`	profondeur maximale	—
`n_calls`	appels de fonction	`CALL`
`n_returns`	retours	`RETURN`

Table 4.2 – Composantes du profil structurel utilisé par l'EIIA.

La distance euclidienne entre les profils de deux programmes est définie par :

$$d(V_A, V_B) = \|V_A - V_B\|_2. \quad (4.2)$$

Cette distance est normalisée par la norme maximale des deux profils :

$$d_{norm}(A, B) = \frac{d(V_A, V_B)}{\max\left(\|V_A\|_2, \|V_B\|_2, \varepsilon\right)}, \qquad \varepsilon = 10^{-12}. \quad (4.3)$$

Le score EIIA est alors obtenu par :

$$S_{\text{EIIA}}(A, B) = \max\left(0,\ 1 - d_{norm}(A, B)\right). \quad (4.4)$$

La figure 4.2 illustre cette chaîne de calcul. Aucune pondération explicite n'est appliquée aux composantes. Cette décision ne signifie toutefois pas qu'elles ont la même influence : les composantes conservant leurs échelles brutes, les effectifs de nœuds peuvent dominer les composantes moins fréquentes, comme la profondeur ou le nombre de retours. Cette limite est prise en compte dans l'interprétation de l'EIIA.

Cette approche s'inscrit dans la famille des métriques statiques qui décrivent la taille et la complexité d'un programme sans nécessiter l'alignement de deux arbres ou graphes [28]. Des travaux consacrés à l'évaluation de la difficulté et de l'effort de programmation montrent également que des métriques telles que la taille du code et la complexité cyclomatique peuvent être corrélées à un indice d'effort interne, l'*Index of Internal Effort* (IIE) [6]. Cette observation soutient l'utilisation de profils quantitatifs comme indicateurs de complexité, sans établir pour autant une équivalence entre l'EIIA et l'IIE. L'EIIA constitue ainsi un indicateur de la proximité globale d'architecture et de complexité, sans prétendre décrire à lui seul la stratégie algorithmique détaillée : deux programmes ayant les mêmes effectifs de nœuds obtiennent un score maximal même si ces nœuds sont agencés différemment.

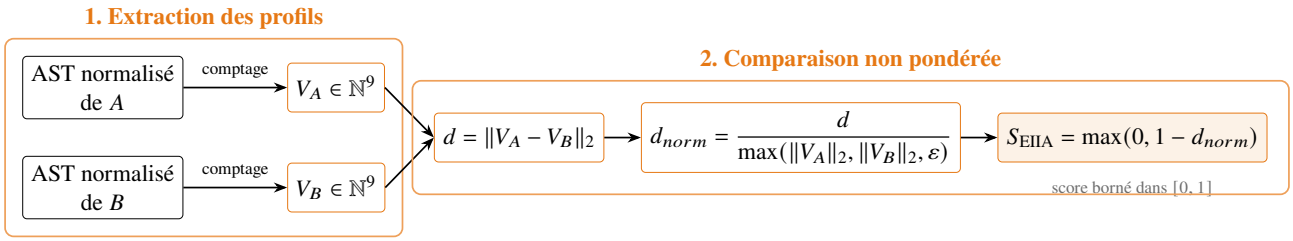


Figure 4.2 – Chaîne de calcul de l'EIIA : extraction de deux profils structurels, puis distance euclidienne normalisée.

4.3 AST normalisé et distance TSED

La deuxième approche évalue la structure grammaticale détaillée du code. Le code Python est d'abord analysé avec Tree-sitter, puis converti en un AST simplifié dont les nœuds portent des types unifiés (`FUNCTION`, `ASSIGN`, `IF`, `FOR`, `BINARY_OP`, `CALL`, `IDENTIFIER`, etc.). Une phase de normalisation réduit ensuite l'influence des choix de style et des détails formels qui ne modifient pas la logique de résolution.

4.3.1 Conversion du CST en AST simplifié

La conversion parcourt les nœuds nommés du CST et associe à chaque type Tree-sitter un type de l'AST interne au moyen d'une table de correspondance. Les nœuds anonymes (mots-clés, ponctuation) ne sont pas conservés, à l'exception des opérateurs, qui sont recopiés comme valeur du nœud (`BINARY_OP(+)`, `COMPARE(==)`, `UNARY_OP(not)`).

Une table de correspondance ne couvre jamais toute la grammaire du langage, et le traitement des types absents influe directement sur les mesures. Leur attribuer un label générique commun, par exemple `UNKNOWN`, rendrait identiques des constructions sans rapport : pour le TSED, une compréhension de liste, une négation `not`, une tranche `l[1:3]` ou une définition de classe deviendraient des nœuds substituables à coût nul. Sur le corpus GitHub, ces constructions sont fréquentes : on y compte notamment 23 compréhensions de liste, 13 définitions de classe, 8 négations et 7 tranches.

Plutôt que de chercher à couvrir toute la grammaire de Python, la conversion applique quatre règles simples (tableau 4.3) :

— **types ajoutés à la table** : une construction ne reçoit son propre type que si elle traduit un choix de résolution. Par exemple, filtrer une liste avec une boucle `for` ou avec la compréhension `[x for x in liste if x % 2 == 0]` sont deux démarches différentes : la compréhension reçoit donc le type `LIST_COMP`, pour que les mesures puissent les distinguer. Son `for` et son `if` sont en revanche typés `FOR` et `IF`, comme une boucle et un test ordinaires, puisqu'ils réalisent eux aussi une itération et une sélection. Le même raisonnement s'applique à la négation (`not x`), aux tranches (`l[1:]`) et aux classes. Les tuples (`a, b`) et les paires clé-valeur (`k: v`) reçoivent aussi un type, afin que les valeurs qu'ils regroupent ne se mélangent pas avec les nœuds voisins ;

— **feuilles** : une chaîne de caractères et un import ne gardent aucun enfant, quel que soit leur découpage interne par l'analyseur. Toute instruction qui commence par `import` ou `from`, quelle que soit sa forme (alias, nom pointé, import relatif ou `*`), devient un unique nœud `IMPORT` : seule la présence de l'import compte, pas le module importé. Le NCST applique déjà ce principe, chaque import y formant un seul nœud `STATEMENT` ;

— **paramètres réduits à leur nom** : quelle que soit sa forme (`n=2`, `n: int`, `*args`, `**kwargs`), un

paramètre devient un nœud `IDENTIFIER`. Le nombre et l'ordre des paramètres sont conservés, ce qui suffit à l'anonymisation en `PARAM_1`, `PARAM_2`, etc. ; les valeurs par défaut et les annotations de type relèvent du style et non de la stratégie ;

— **suppression des nœuds non reconnus** : un nœud dont le type est absent de la table n'est pas conservé et ses enfants prennent directement sa place. Les constructions rares (`with ... as`, `match`, décorateurs, etc.) gardent ainsi leur contenu (appels, identifiants, blocs) sans introduire d'étiquette artificielle. Par exemple, pour une fonction décorée par `@cache`, l'AST contient directement `IDENTIFIER(cache)` suivi de `FUNCTION`. Les déclarations `global` et `nonlocal` sont ignorées.

La suppression d'un nœud non reconnu préserve l'ordre de l'arbre : ses enfants sont insérés exactement à sa position, entre ses voisins, qui ne sont pas déplacés. La règle s'applique récursivement, si bien que plusieurs niveaux non reconnus successifs disparaissent ensemble. Dans l'exemple de la figure 4.3, chaque appel de l'instruction `with` est emboîté dans trois nœuds intermédiaires qui disparaissent ensemble ; le contenu et son ordre sont conservés, ce qui importe pour le TSED, et les instructions voisines `a = 1` et `return a` restent à leur place.

Code

```
def f(p):
    a = 1
    with open(p) as fh, \
         open("y") as g:
        b = 2
    return a
```

CST Tree-sitter (extrait)

```
block                          BLOCK
  assignment          a = 1      ASSIGN       a = 1
  with_statement                 WITH
    with_clause     (x)            CALL       open(p)
      with_item     (x)            IDENTIFIER(fh)
        as_pattern  (x)            CALL       open("y")
          call        open(p)      IDENTIFIER(g)
          identifier  fh           BLOCK      b = 2
      with_item     (x)          RETURN       return a
        as_pattern  (x)
          call        open("y")
          identifier  g
    block             b = 2
  return_statement    return a
```

AST obtenu

Figure 4.3 – Suppression des nœuds non reconnus, marqués `(x)` : leurs enfants prennent leur place dans le même ordre, et les instructions voisines ne sont pas déplacées. Le nœud `as_pattern_target` qui entoure `fh` et `g` est omis de l'extrait ; il est supprimé de la même manière.

Avec ces règles, l'AST ne contient aucun nœud `UNKNOWN`, par construction. Les conséquences de la suppression des nœuds non reconnus sont discutées dans les limites (section 6.2).

Pour l'EIIA (tableau 4.2), le `for` et le `if` d'une compréhension sont donc comptés comme une itération et une sélection, et la négation `not`, typée `UNARY_OP`, est comptée parmi les opérateurs.

4.3.2 Normalisation de l'AST

La normalisation transforme les deux arbres avant le calcul de la distance d'édition. Elle comprend les opérations suivantes :

— l'anonymisation canonique des paramètres, des variables locales et des fonctions du fichier, remplacés dans l'ordre d'apparition par `PARAM_1`, `VAR_1` et `FUNC_1` ;

Règle	Effet sur l'AST (exemple de code)
Types ajoutés	`LIST_COMP`, `DICT_COMP`, `SET_COMP`, `GENERATOR`, dont le `for` et le `if` sont typés `FOR` et `IF` (`[x for x in l if c]`) ; `UNARY_OP(not)` ; `SLICE` (`l[1:3]`) ; `PAIR` (`k: v`) ; `TUPLE` (`a, b = b, a`) ; `CLASS`
Feuilles	`STRING` sans enfant ; `IMPORT` sans enfant pour `import os.path as osp`, `from math import *`, etc.
Paramètres	`def f(l, n: int = 2, *a, **k)` → `PARAMETERS(l, n, a, k)`, y compris pour une `lambda`
Nœuds supprimés	type absent de la table : remplacé par ses enfants ; `global` et `nonlocal` : ignorés

TABLE 4.3 – Règles de conversion du CST Tree-sitter en AST interne. Aucun nœud `UNKNOWN` n'est produit.

— l'abstraction des littéraux, qui remplace les valeurs concrètes par les catégories `NUM`, `STR`, `BOOL` et `NULL` ;

— l'expansion des affectations augmentées, qui réécrit `a += b` sous la forme canonique `a = a + b` ;

— l'*inlining* des fonctions auxiliaires simples, limité aux fonctions sûres : un seul `return` portant une expression unique, sans boucle, condition, appel, exception ni récursion.

Les opérateurs, tels que +, % ou ==, ainsi que les structures de contrôle sont conservés, car ils portent une information importante sur la stratégie de résolution. Les commentaires sont supprimés dès la conversion de l'arbre. La figure 4.4 illustre l'effet combiné de ces règles sur une instruction.

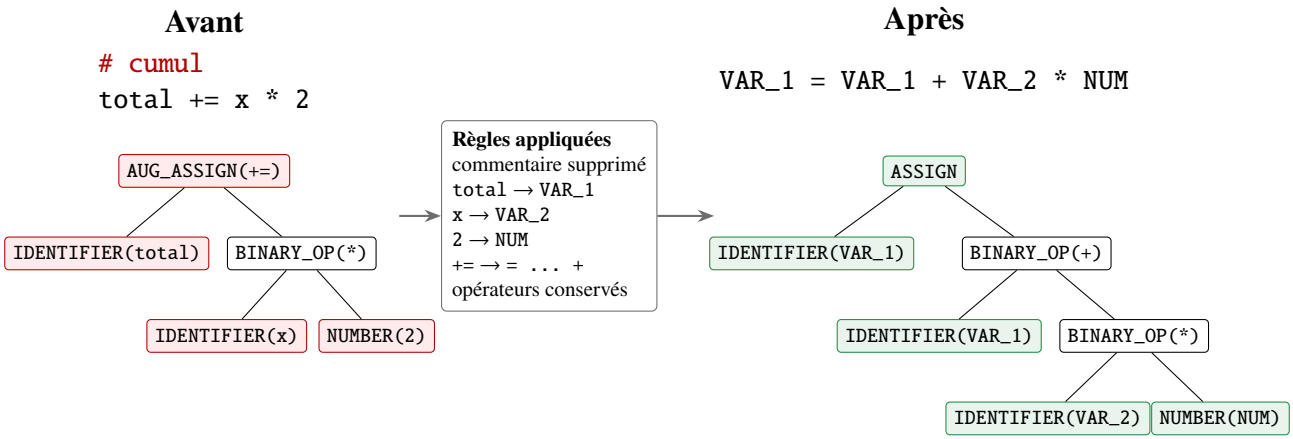


FIGURE 4.4 – Normalisation d'une instruction. Les nœuds en rouge portent des détails de surface (noms, valeurs, sucre syntaxique) ; après normalisation, ils sont remplacés par des formes canoniques (en vert). Les opérateurs sont conservés.

4.3.3 Distance d'édition entre arbres

La distance TSED correspond à la distance d'édition entre les deux arbres normalisés, calculée selon l'algorithme de Zhang et Shasha [31]. Chaque nœud est décrit par un label associant son type à sa valeur résiduelle, par exemple `BINARY_OP(+)` ou `IDENTIFIER(VAR_1)`. Les coûts d'insertion, de

suppression et de substitution sont unitaires :

$$c_{ins} = c_{del} = 1, \qquad c_{sub}(u, v) = \begin{cases} 0 & \text{si label}(u) = \text{label}(v), \\ 1 & \text{sinon.} \end{cases} \tag{4.5}$$

Si $TED(A, B)$ désigne la distance d'édition et $|A|$ et $|B|$ le nombre de nœuds des deux arbres, le score est défini par :

$$S_{\text{TSED}}(A, B) = 1 - \frac{TED(A, B)}{\max(|A|, |B|)}. \tag{4.6}$$

La figure 4.5 montre un cas minimal : deux fonctions qui ne diffèrent que par un opérateur. Une seule substitution suffit à transformer un arbre en l'autre.

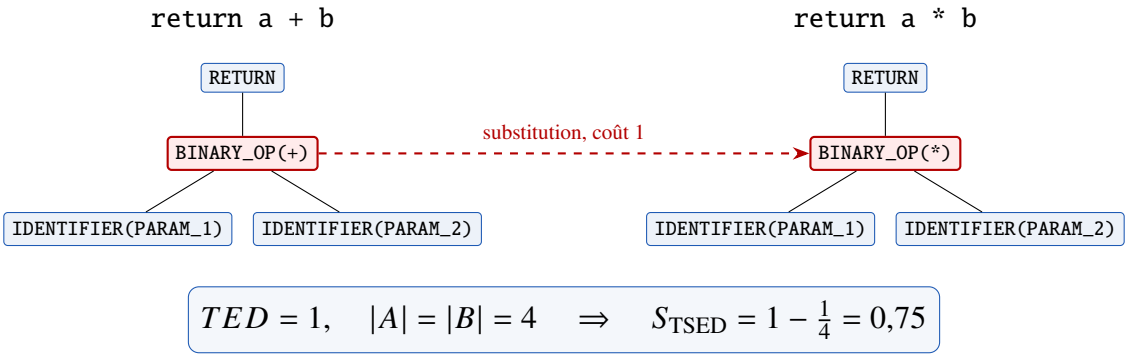


Figure 4.5 – Exemple de calcul TSED sur deux AST normalisés de quatre nœuds.

Dans le cas de programmes modulaires, le système sélectionne, pour le calcul du TSED, la fonction dont l'AST contient le plus de nœuds (la première en cas d'égalité) afin de comparer en priorité la fonction centrale de la correction. L'EIIA et le NCST restent calculés sur la représentation prévue par leur propre pipeline ; cette sélection ne doit donc pas être interprétée comme une réduction commune des trois mesures.

4.4 Arbre structurel normalisé (NCST)

La troisième approche compare l'organisation structurelle abstraite des programmes. Deux AST peuvent être très différents tout en présentant une organisation proche lorsqu'ils partagent les mêmes catégories de blocs, aux mêmes niveaux d'imbrication et dans le même ordre.

4.4.1 Construction et normalisation du NCST

Le NCST (*Normalized Control Structure Tree*) est d'abord construit sous forme brute directement à partir du CST produit par Tree-sitter. Les instructions sont regroupées en séquences et chaque construction structurante devient un nœud typé, relié à ses blocs par des arêtes étiquetées.

Une normalisation structurelle est ensuite appliquée. Elle supprime toutes les informations dépendantes du code source :

— les noms des variables ;

— les valeurs constantes ;

— les expressions ;
— les opérateurs ;
— les paramètres ;
— les noms exacts des fonctions.

Seules les catégories et les relations structurelles sont conservées (tableau 4.4). Contrairement à une simple extraction des structures de contrôle, les instructions ordinaires restent présentes sous la forme de nœuds anonymes `STATEMENT` : le NCST normalisé conserve donc la présence et la position des instructions, sans leur contenu.

Catégories conservées	`PROGRAM`, `CLASS`, `FUNCTION`, `SEQUENCE`, `ITERATION`, `SELECTION`, `EXCEPTION`, `HANDLER`, `CONTEXT`, `COMPREHENSION`, `LAMBDA`, `TRANSFER`, `STATEMENT`
Relations conservées	`BODY`, `CONTAINS`, `NEXT`, `THEN`, `ELSE`, `ELIF`, `CASE`, `DEFAULT`, `TRY`, `CATCH`, `FINALLY`
Informations supprimées	noms de variables, constantes, expressions, opérateurs, paramètres, noms exacts des fonctions

Table 4.4 – Vocabulaire du NCST normalisé.

4.4.2 Niveau structurel

Chaque nœud reçoit un niveau structurel (*level*) qui représente son niveau d'imbrication. Les relations qui ouvrent un bloc (`BODY`, `THEN`, `ELSE`, `ELIF`, `CASE`, `DEFAULT`, `TRY`, `CATCH`, `FINALLY`) augmentent le niveau ; les relations `CONTAINS` et `NEXT` conservent le niveau courant (figure 4.6). Le niveau structurel n'est donc pas la profondeur de l'arbre : un nœud `FUNCTION` contenu dans la séquence racine reste au même niveau que cette séquence.

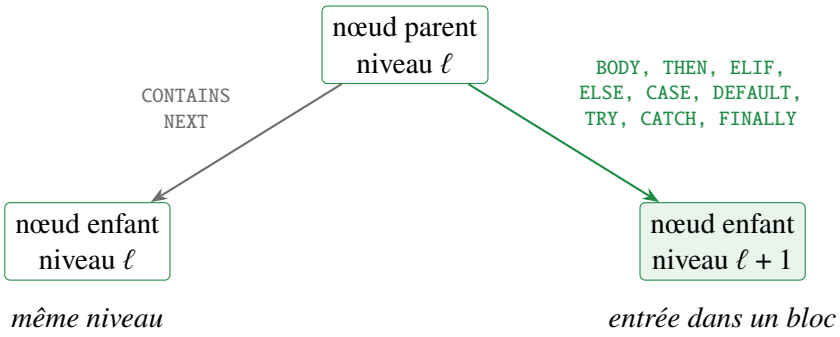


Figure 4.6 – Calcul du niveau structurel dans le NCST : seules les arêtes qui ouvrent un bloc incrémentent le niveau.

La normalisation conserve les imbrications. Une sélection placée dans une itération et une itération placée dans une sélection produisent deux arbres distincts (figure 4.7) : la normalisation ne confond pas ces deux situations.

4.4.3 Exemple de NCST normalisé et linéarisation

Pour comparer deux NCST, chaque arbre est parcouru en profondeur depuis sa racine. Les nœuds sont placés dans une liste qui conserve l'ordre du parcours, chaque élément étant le couple $(kind, level)$.

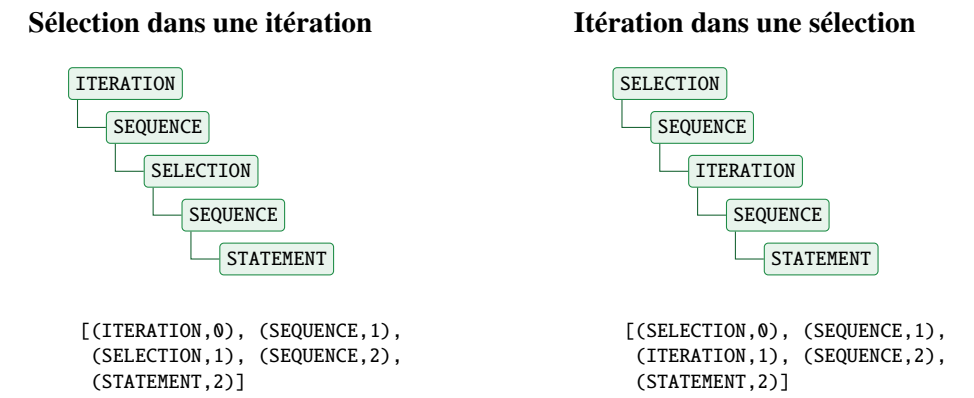


Figure 4.7 – Deux imbrications inverses conduisent à deux NCST normalisés différents (niveaux relatifs, la racine du fragment étant au niveau 0). L'appariement de la section 4.4.5 n'associe que `SEQUENCE`, `SEQUENCE` et `STATEMENT` : $M = 3$ et $S_{\text{NCST}} = 6/10 = 0{,}6$.

La figure 4.8 présente le NCST normalisé d'une fonction contenant deux instructions, une boucle, une sélection dans cette boucle et une instruction `return`. Les noms de variables, les valeurs et les opérateurs ont disparu : il ne reste que la structure abstraite du programme.

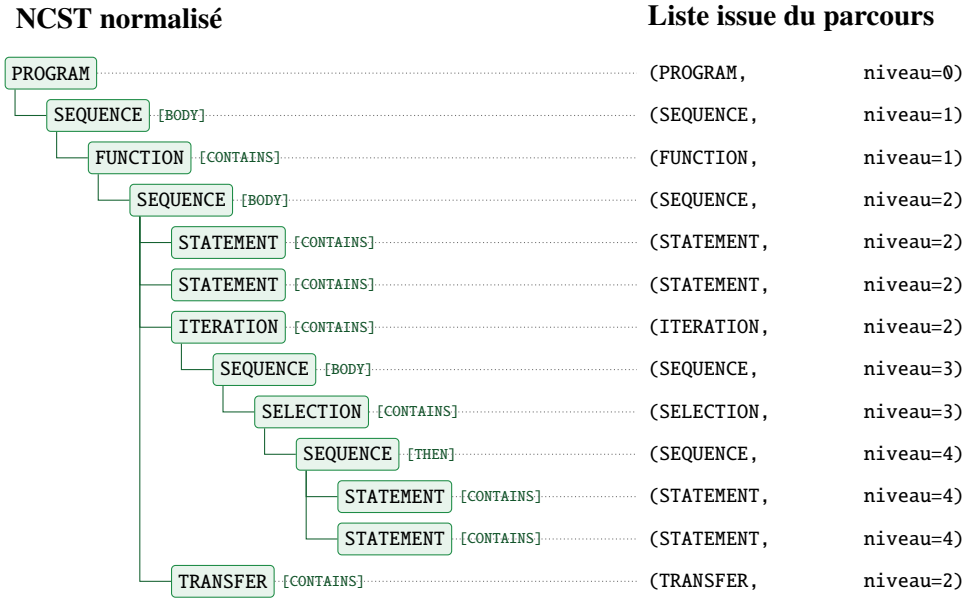


Figure 4.8 – NCST normalisé d'une fonction (gauche), avec entre crochets la relation qui relie chaque nœud à son parent, et liste (*kind*, *level*) obtenue par parcours en profondeur (droite). Le retrait traduit la profondeur dans l'arbre, tandis que le niveau n'augmente que sur les arêtes `BODY` et `THEN`.

4.4.4 Invariance du squelette structurel

La figure 4.9 reprend deux fonctions dont les expressions et les valeurs manipulées sont différentes. Le premier programme recherche un nombre pair, tandis que le second recherche une valeur cible. Malgré cette différence de domaine, les deux solutions parcourent une collection, testent une condition puis retournent immédiatement un résultat en cas de succès, avant d'effectuer un retour par défaut. Après normalisation, les deux NCST produisent la même liste.

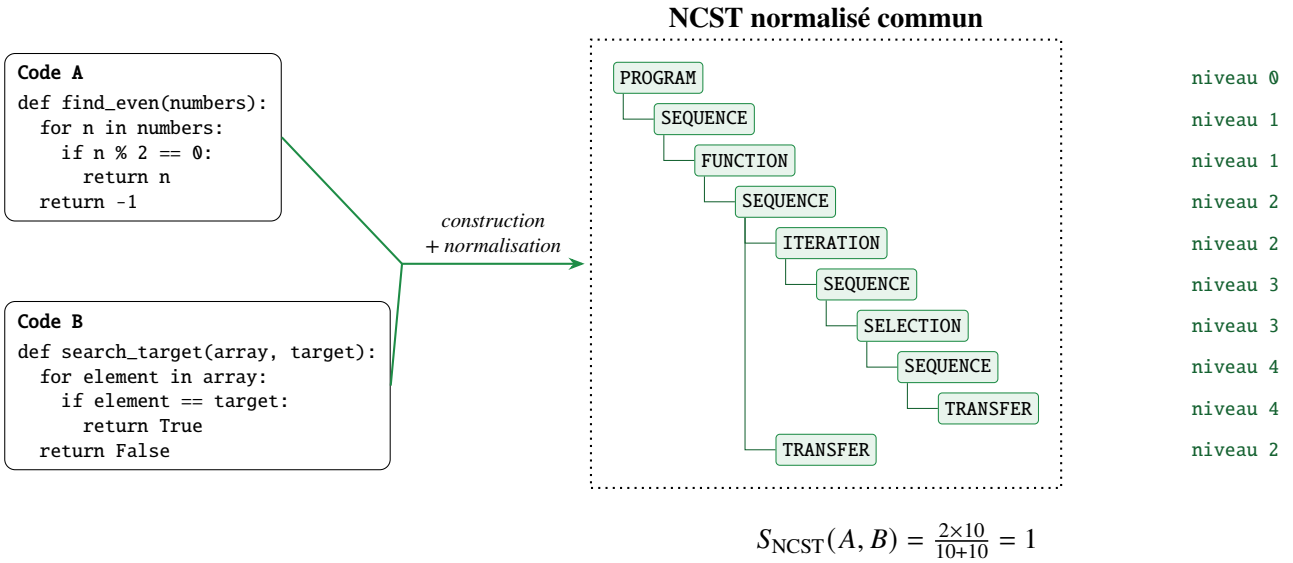


Figure 4.9 – Invariance du NCST normalisé sur deux programmes différents. Les codes A et B utilisent des variables, des opérateurs et des valeurs de retour distincts, mais leur normalisation produit la même liste de nœuds abstraits.

Dans cet exemple, le score $S_{\text{NCST}} = 1$ exprime une correspondance parfaite de la structure conservée par la représentation. Il ne signifie pas que les deux fonctions sont équivalentes du point de vue de leur résultat, mais qu'elles partagent une même organisation : itération, test, retour anticipé et retour par défaut.

4.4.5 Algorithme d'appariement naïf ordonné

Choix du petit et du grand arbre. Le nombre de nœuds de chaque NCST normalisé est compté. Le NCST qui contient le moins de nœuds devient le *petit arbre A*, celui qui en contient le plus devient le *grand arbre B*. Le matching est effectué du petit arbre vers le grand arbre, ce qui évite de considérer chaque nœud du grand arbre comme point de départ possible.

Principe. L'algorithme est volontairement naïf : il ne s'agit pas d'un calcul de distance d'édition d'arbres. Pour chaque nœud du petit arbre :

1. le grand arbre est parcouru à partir de la position qui suit le dernier nœud apparié ;
2. le premier nœud compatible est recherché ;
3. deux nœuds sont compatibles s'ils possèdent le même type abstrait (*kind*) et le même niveau structurel (*level*) ;
4. lorsqu'un appariement est trouvé, sa position dans le grand arbre est mémorisée ;
5. le nœud suivant du petit arbre ne peut être recherché qu'après cette position.

La condition de compatibilité s'écrit :

$$\text{compatible}(a, b) \iff a.kind = b.kind \ \wedge \ a.level = b.level. \tag{4.7}$$

La contrainte d'ordre est assurée par l'indice de reprise $last_match_index_b + 1$: un nœud du grand arbre ne peut pas être utilisé deux fois et les appariements restent dans l'ordre du parcours. Un nœud du petit arbre sans correspondant ne fait pas avancer cet indice. La méthode principale est la suivante :

```
matches = []
last_match_index_b = -1

for node_a in nodes_a:                          # petit arbre
    for i in range(last_match_index_b + 1, len(nodes_b)):
        node_b = nodes_b[i]                     # grand arbre
        if node_a.kind == node_b.kind and node_a.level == node_b.level:
            matches.append((node_a, node_b))
            last_match_index_b = i
            break
```

Exemple d'appariement. La figure 4.10 applique cet algorithme à un petit arbre de cinq nœuds et à un grand arbre de six nœuds. Lors de la recherche de `ITERATION` (niveau 2), le parcours du grand arbre commence à l'indice 2 : le `STATEMENT` de niveau 2 n'est pas compatible et il est dépassé. Ce nœud supplémentaire reste non apparié.

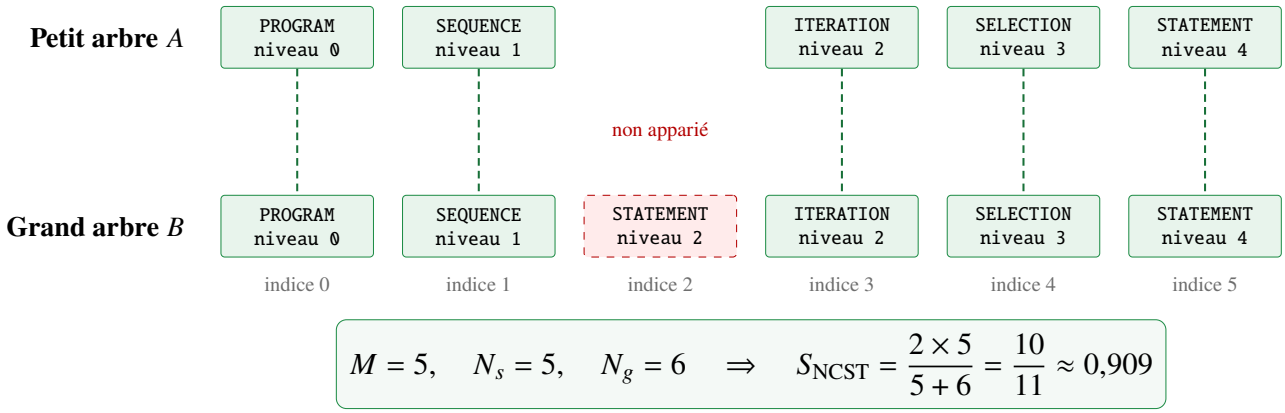


Figure 4.10 – Appariement ordonné du petit arbre vers le grand arbre. Les pointillés relient des nœuds de même *kind* et de même *level*, dans un ordre croissant d'indices.

4.4.6 Calcul de la similarité NCST

Soit M le nombre de nœuds appariés, N_s le nombre de nœuds du petit arbre et N_g le nombre de nœuds du grand arbre. La similarité NCST utilise l'indice de Sørensen-Dice :

$$S_{\mathrm{NCST}}(A, B) = \frac{2M}{N_s + N_g}. \tag{4.8}$$

Comme $M \leq N_s \leq N_g$, le score est compris entre 0 et 1 : il vaut 0 lorsqu'aucun nœud n'est apparié et 1 lorsque tous les nœuds des deux arbres sont appariés, ce qui impose $N_s = N_g = M$. Une valeur intermédiaire indique que certains nœuds sont communs, mais que l'un des arbres contient des nœuds supplémentaires ou différents. Par exemple, avec $N_s = 2$, $N_g = 4$ et $M = 2$:

$$S_{\mathrm{NCST}} = \frac{2 \times 2}{2+4} = \frac{4}{6} \approx 0{,}6667. \tag{4.9}$$

Même lorsque tous les nœuds du petit arbre trouvent un correspondant, la différence de taille entre les deux arbres limite donc le score.

4.4.7 Positionnement par rapport au TSED et à l'EIIA

Les trois mesures ont des rôles différents. Le TSED compare finement les AST au moyen d'une distance d'édition d'arbres et tient compte des labels et de leur structure détaillée. L'EIIA compare un vecteur de métriques structurelles (nombre de nœuds, d'opérateurs, d'affectations, de sélections, d'itérations, d'appels et de retours). Le NCST, enfin, compare uniquement des nœuds abstraits normalisés à travers leur *kind* et leur *level*, en ignorant les noms, les valeurs et les expressions, mais en conservant l'ordre général du parcours. Le NCST n'est donc pas un *Tree Edit Distance* : il s'agit d'un appariement ordonné et naïf de nœuds abstraits. Le score obtenu est une heuristique de proximité structurelle.

4.4.8 Limites de l'approche

Cette première version est volontairement simple. Elle ne vérifie pas directement :

— le parent exact du nœud ;
— la relation d'arête (`BODY`, `THEN`, `NEXT`, etc.) ;
— l'égalité complète des sous-arbres ;
— la signification des instructions `STATEMENT` ;
— les labels ou les expressions supprimés pendant la normalisation.

Par ailleurs, l'appariement est glouton : chaque nœud du petit arbre est associé au premier nœud compatible rencontré, sans retour en arrière. Ce choix garantit un calcul simple, dont le coût reste au pire de l'ordre de $N_s \times N_g$ comparaisons, mais il ne cherche pas l'appariement maximal possible. Deux arbres peuvent ainsi obtenir un score NCST élevé lorsqu'ils contiennent les mêmes catégories abstraites aux mêmes niveaux, même si leurs parents ou leurs relations exactes diffèrent. Cette mesure doit donc être interprétée comme un indicateur de proximité structurelle abstraite, et non comme une preuve d'équivalence sémantique ou de plagiat.

4.5 Exécution et synthèse des mesures

Le pipeline ne fusionne pas les trois mesures. Elles décrivent des dimensions distinctes de la similarité logicielle, et leur combinaison prématurée masquerait la contribution de chaque représentation. Pour chaque paire de programmes, le système exporte donc séparément les trois scores S_{EIIA}, S_{TSED} et S_{NCST}, selon le format présenté dans le tableau 4.5.

source_a	source_b	tsed_similarity	eiia_similarity	ncst_similarity	error
ex_A.py	ex_B.py	S_{TSED}	S_{EIIA}	S_{NCST}	—
ex_A.py	ex_C.py	S_{TSED}	S_{EIIA}	S_{NCST}	—
			...		

Table 4.5 – Structure du fichier CSV exporté par le pipeline : une ligne par paire, une colonne par mesure, et une colonne d'erreur en cas d'échec de l'analyse.

Cette sortie conserve une information interprétable sur la nature de la proximité. Le tableau 4.6 donne quelques profils de lecture typiques : par exemple, deux exercices peuvent obtenir un score EIIA élevé, signe d'une architecture globale comparable, tout en présentant un score NCST faible, indiquant une organisation structurelle différente.

EIIA	TSED	NCST	Lecture possible
élevé	élevé	élevé	Corrections très proches, à la fois en taille, en syntaxe et en structure.
élevé	faible	élevé	Même organisation structurelle et même complexité, mais écritures détaillées différentes.
faible	faible	élevé	Même organisation appliquée à des expressions de tailles différentes.
élevé	—	faible	Complexité comparable mais organisations structurelles distinctes.

Table 4.6 – Exemples de profils de scores et de leur interprétation.

Les trois colonnes permettent ainsi d'analyser séparément les dimensions de la substituabilité algorithmique avant toute interprétation pédagogique.

4.6 Interface de comparaison Streamlit

Une interface web développée avec Streamlit rend le pipeline utilisable sans passer par la ligne de commande. Elle ne réimplémente pas les mesures : elle prépare un jeu de fichiers, appelle la fonction `run_pipeline` du module `run_poc`, lit le CSV produit, puis affiche les scores et les arbres intermédiaires. La figure 4.11 résume son fonctionnement.

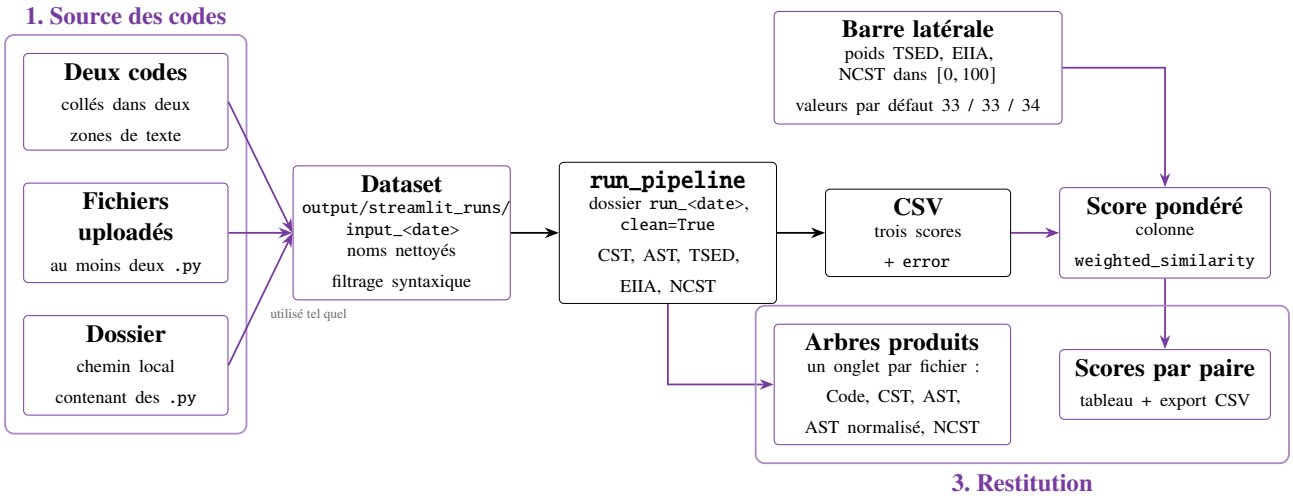


Figure 4.11 – Fonctionnement de l'interface Streamlit : préparation du jeu de fichiers, appel du pipeline, pondération facultative et restitution des scores et des arbres.

4.6.1 Préparation des codes à comparer

L'utilisateur choisit l'une des trois sources proposées :

— **Deux codes** : deux zones de texte reçoivent les programmes et leurs noms de fichier. Ils sont écrits dans un dossier temporaire unique `output/streamlit_runs/input_<date>` ;

— **Fichiers uploadés** : au moins deux fichiers `.py` sont requis, puis copiés dans un dossier temporaire du même type ;

— **Dossier** : un chemin local est fourni ; l'interface vérifie qu'il contient au moins deux fichiers `.py`, condition nécessaire pour former des paires, et affiche le nombre de fichiers détectés.

Les noms fournis par l'utilisateur passent par la fonction `safe_python_name`, qui ne garde que le nom de base du fichier et ajoute l'extension `.py` si elle manque. Cette précaution empêche un nom uploadé contenant un chemin d'écrire en dehors du dossier temporaire. Chaque dossier créé par l'interface est nommé d'après la date et l'heure du test, par exemple `run_2026-10-15_14h32m05s`, ce qui permet de retrouver facilement les résultats d'une session. Si un dossier du même nom existe déjà, un suffixe numérique est ajouté : deux comparaisons successives n'écrasent donc jamais leurs fichiers.

4.6.2 Filtrage des fichiers syntaxiquement invalides

Tree-sitter est un analyseur tolérant aux erreurs : il ne lève aucune exception sur un code invalide et produit toujours un arbre complet. Les portions qu'il ne parvient pas à rattacher à la grammaire sont regroupées dans des nœuds `ERROR`, et les jetons manquants, comme un « `:` » oublié, sont insérés sous forme de nœuds `MISSING` de longueur nulle. L'attribut `has_error` de la racine indique si l'arbre contient au moins l'un de ces nœuds.

Sans filtrage, un tel fichier traverse tout le pipeline et reçoit des scores qui ne reflètent pas sa structure. Par exemple, si l'en-tête `def f(a, b)` est privé de ses deux-points, la fonction entière est absorbée dans un nœud `ERROR` : l'AST ne contient plus de nœud `FUNCTION`, le fichier est traité comme un module unique et, le nœud `ERROR` étant supprimé comme tout nœud non reconnu (section 4.3.1), son contenu se réduit à une suite de fragments placés directement sous la racine : identifiants isolés, `x = a` réduit à deux identifiants, `if x >` réduit à `x`. Le NCST se réduit quant à lui à un seul nœud `STATEMENT` contenant le texte brut.

Avant de lancer le pipeline, l'interface analyse donc chaque fichier avec Tree-sitter. Les fichiers dont la racine signale une erreur sont écartés de la comparaison, et seuls les fichiers valides sont copiés dans un dossier dédié `valid_<date>` transmis à `run_pipeline`. La liste des fichiers écartés est affichée au-dessus du tableau des scores. Si moins de deux fichiers valides subsistent, la comparaison est interrompue avec un message d'erreur. Le dossier source choisi par l'utilisateur n'est jamais modifié.

Ce contrôle ne porte que sur la syntaxe. Le pipeline réalise une analyse statique : il n'exécute ni n'importe jamais le code comparé. Un programme syntaxiquement correct mais qui échouerait à l'exécution (module importé absent, variable non définie, erreur de type) produit donc des représentations complètes et est comparé normalement. Ce comportement est cohérent avec l'objectif du travail, qui porte sur la démarche de résolution et non sur la correction fonctionnelle des programmes. Enfin, le filtrage est propre à l'interface : l'exécution en ligne de commande de `run_poc.py` compare tous les fichiers du dossier.

4.6.3 Exécution et pondération des scores

Le bouton « Comparer les paires » lance `run_pipeline` dans un dossier de sortie dédié `run_<uuid>`, avec l'option `clean=True`. Les résultats, le dossier d'exécution et le dataset sont conservés dans `st.session_state`, ce qui évite de relancer le calcul à chaque interaction avec la page. En cas d'erreur, l'exception est affichée dans l'interface.

La barre latérale propose trois curseurs, de 0 à 100 par pas de 1, pour les poids w_T, w_E et w_N associés respectivement au TSED, à l'EIIA et au NCST. Les valeurs par défaut sont 33, 33 et 34. Le score pondéré ajouté à chaque paire est :

$$S_{\text{pond}}(A, B) = \frac{w_T\, S_{\text{TSED}}(A, B) + w_E\, S_{\text{EIIA}}(A, B) + w_N\, S_{\text{NCST}}(A, B)}{w_T + w_E + w_N}. \tag{4.10}$$

Le calcul est refusé si la somme des poids est nulle. Si l'un des trois scores est absent ou non numérique, par exemple lorsqu'une erreur d'analyse est survenue, la colonne `weighted_similarity` reste vide pour cette paire.

Ce score pondéré est un outil d'exploration et non une mesure supplémentaire du pipeline. Les poids sont choisis par l'utilisateur ; ils ne sont ni appris ni calibrés sur des jugements d'enseignants. Les trois scores restent d'ailleurs affichés séparément à côté de S_{pond}, ce qui préserve la lecture multi-vues défendue dans la section précédente.

4.6.4 Restitution des résultats

Les résultats sont présentés dans un tableau qui reprend les colonnes du CSV du pipeline, complétées par `weighted_similarity`. Un bouton permet de télécharger ce tableau au format CSV.

L'interface affiche ensuite les arbres produits pour chaque fichier source, dans un onglet par fichier (tableau 4.7). Les arbres CST, AST et AST normalisé sont lus dans le cache du pipeline. Le NCST brut et le NCST normalisé sont reconstruits à l'affichage avec `PythonNCSTBuilder` et `NCSTNormalizer`, puis enregistrés au format texte et JSON dans les dossiers `cache/ncst` et `cache/normalized_ncst`. Si un artefact attendu est absent, un avertissement est affiché à la place.

Onglet	Contenu affiché
Code	Code Python source du fichier
CST	Arbre syntaxique concret Tree-sitter (`cache/tree_sitter/*.cst.txt`)
AST	AST simplifié (`cache/ast/*.ast.txt`)
AST normalisé	AST après anonymisation, abstraction des littéraux, expansion et *inlining* (`cache/normalized_ast/*.normalized.txt`)
NCST	Deux sous-onglets : « NCST brut » et « NCST normalisé », rendus textuellement par `render_text`

TABLE 4.7 – Onglets d'inspection proposés pour chaque fichier source.

Cette inspection relie chaque score aux représentations qui l'ont produit : lorsqu'un score NCST est surprenant, l'utilisateur peut comparer directement les deux NCST normalisés et identifier les nœuds qui ne trouvent pas de correspondant.

4.7 Protocole d'évaluation des approches

L'évaluation se déroule en deux phases, résumées par la figure 4.12. Les trois scores sont d'abord calculés sur des variantes contrôlées, obtenues en appliquant une transformation identifiée à une correction de référence. Cette étape permet d'observer séparément la sensibilité de chaque mesure à un renommage, à une modification de structure de contrôle ou à une transformation de l'organisation du code. Les résultats sont ensuite produits sur le corpus complet afin d'étudier la distribution des scores et d'identifier les paires candidates pour une analyse pédagogique. L'interface Streamlit sert de support à cette analyse : elle permet de rejouer une paire et d'inspecter ses arbres intermédiaires.

Les mesures ne sont pas interprétées comme une vérité terrain automatique. La comparaison porte d'abord sur leur comportement attendu face aux variantes, puis sur leur complémentarité. Les cas où

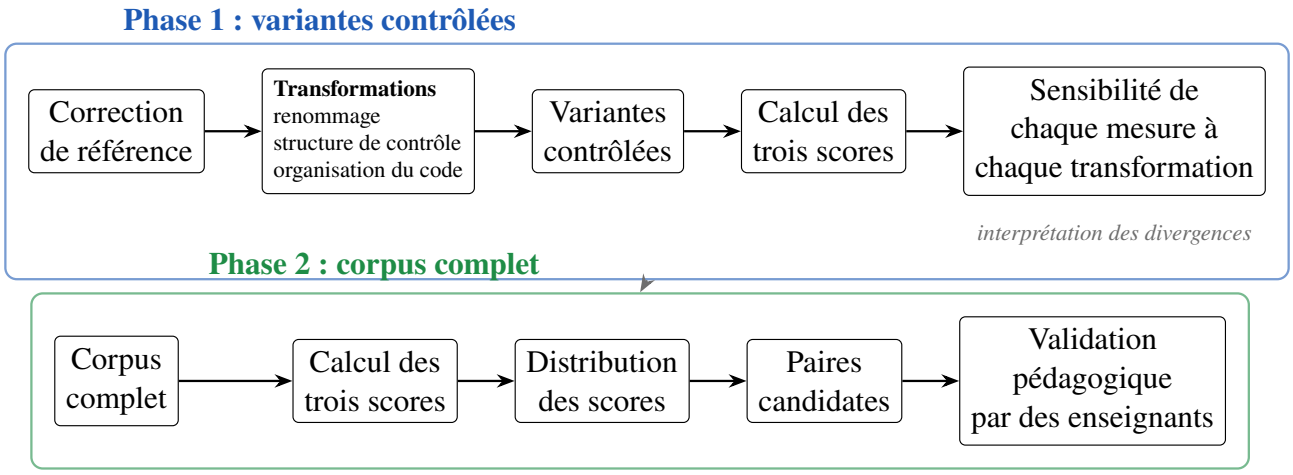


FIGURE 4.12 – Protocole d'évaluation en deux phases.

les scores divergent sont conservés comme des situations informatives : ils permettent d'identifier la dimension de la similarité capturée par chaque approche et de préparer une validation ultérieure par des enseignants, seuls à même de juger si deux exercices sont réellement substituables d'un point de vue pédagogique.

Chapitre 5

Résultats expérimentaux

Ce chapitre présente le corpus, les variantes contrôlées et les résultats du pipeline. Les trois mesures sont analysées séparément afin d'identifier la dimension de similarité capturée par l'EIIA, le TSED et le NCST. En l'absence d'annotations pédagogiques exhaustives, les scores sont interprétés comme des indicateurs de comparaison et non comme une vérité terrain.

5.1 Constitution du corpus et variantes contrôlées

Le plus grand corpus est issue des travavaux d'A. Ziani, il comprend 906 exercices écrits en Python, chacun associé à un énoncé et à une correction de référence, soit 409 965 paires à comparéeset pour celui avec des variantes controlées il a été faits manuellement et comprend 12 exercices écrits en Python dont l'exercice de base inclu. Les corrections comptent en moyenne 14 lignes . Pour chaque correction, le pipeline extrait les représentations nécessaires au calcul des trois scores. Les erreurs de traitement survenues lors de la comparaison d'une paire sont conservées dans la colonne `error` de l'export CSV.

Tree-sitter ne lève pas d'erreur sur un code invalide (section 4.6.2). Un contrôle syntaxique préalable montre que 100 des 906 corrections du corpus complet sont invalides. Dans 96 cas, le code provient de GitHub et reste encadré par les balises Markdown ` ```python ` et ` ``` ` : il suffit de retirer ces lignes pour obtenir un programme valide, sans modifier le code lui-même. Les quatre autres fichiers ne sont pas des programmes Python exploitables : un code écrit en C, une phrase en anglais placée avant le code, une adresse électronique altérée lors de la récupération de la page web et un littéral numérique invalide.

Les variantes variantes contrôlées sont produites à partir d'une base pris aléatoirement dans les corrections de référence. Chaque variante applique une transformation identifiée : renommage des variables, modification de la syntaxe, changement d'une structure de contrôle, ajout ou suppression d'une fonction auxiliaire, ou modification d'une expression, d'un opérateur ou d'un littéral. Cette construction permet de comparer le comportement attendu des mesures avant l'analyse du corpus complet.

À titre d'illustration, la figure 5.1 met en regard la correction de référence `01_base.py` et la variante `9_while_index.py`. Les deux programmes calculent exactement le même résultat, la somme et le nombre des éléments pairs, et suivent la même démarche : parcourir la liste, tester la parité, mettre à jour deux accumulateurs. Seule la manière de parcourir la liste change. La boucle `for` est remplacée par une boucle `while` pilotée par un indice explicite, ce qui introduit trois instructions absentes de la référence : l'initialisation `i = 0`, l'accès `x = liste[i]` et l'incrémentation `i += 1`. Cette variante est intéressante car elle conserve la stratégie de résolution tout en modifiant sensiblement l'arbre

Métrique	Valeur
Nombre total de fichiers	906
Nombre total de lignes	12 129
Nombre maximum de lignes	216
Nombre moyen de lignes	13,39
Paires comparées	409 965

Table 5.1 – Statistiques descriptives du corpus de corrections de référence.

Métrique	Valeur
Nombre total de fichiers	13
Nombre total de lignes	133
Nombre maximum de lignes	28
Nombre moyen de lignes	11,08
Paires comparées	78

Table 5.2 – Statistiques descriptives du jeu de variantes contrôlées.

syntaxique : une mesure de substituabilité devrait donc la juger proche de la référence, alors qu'une mesure trop sensible à la syntaxe risque de la pénaliser.

`01_base.py` (référence)

```python
def stats_pairs(liste):
    total = 0
    nb = 0
    for x in liste:
        if x % 2 == 0:
            total += x
            nb += 1
    return total, nb
```

`9_while_index.py` (variante)

```python
def stats_pairs(liste):
    total = 0
    nb = 0
    i = 0                  # ajout
    while i < len(liste): # for->while
        x = liste[i]       # ajout
        if x % 2 == 0:
            total += x
            nb += 1
        i += 1             # ajout
    return total, nb
```

Figure 5.1 – Correction de référence et variante contrôlée « `while` avec index » : même stratégie, structure de parcours différente.

5.2 Résultats sur les variantes contrôlées

Le jeu de variantes contrôlées est construit autour d'un exercice simple : calculer la somme et le nombre des éléments pairs d'une liste. La correction de référence `01_base.py` parcourt la liste avec une boucle `for`, teste la parité avec `x % 2 == 0`, puis met à jour deux accumulateurs. Douze variantes en sont dérivées (tableau 5.3). Chacune applique une transformation identifiée, ce qui permet d'attribuer à cette transformation toute variation des scores. Les variantes sont réparties en quatre catégories :

- des **contrôles**, dont le résultat est connu à l'avance ;
- des transformations pour lesquelles une **invariance** est attendue ;
- des réécritures qui conservent la **même stratégie** de résolution ;
- des solutions fondées sur une **stratégie différente**, dont la variante `reduce` sert de témoin négatif structurel.

Les treize fichiers forment 78 paires. L'analyse qui suit porte d'abord sur les douze paires qui comparent la référence à chacune de ses variantes (tableau 5.4). Pour chaque mesure, le comportement attendu est déduit des règles présentées au chapitre précédent, puis confronté aux scores observés.

Catégorie	Variante	Transformation appliquée
Contrôles	11 Copie exacte	Copie identique de la référence
	7 Condition `== 1`	Somme des impairs au lieu des pairs (`x % 2 == 1`)
Invariance	13 Instructions inversées	Ordre des initialisations et des mises à jour inversé
	6 Opérateurs équivalents	`(x & 1) == 0` et affectations écrites sans +=
Même stratégie	4 Condition inversée	Test inversé suivi d'un `continue`
	9 `while` avec index	Boucle `while` sur un indice explicite
	12 Sortie ajoutée	Collecte supplémentaire des indices avec `enumerate`
	5 Code mort	Instructions, condition et boucle jamais exécutées
	8 Fonctions auxiliaires	Test de parité et mise à jour extraits dans deux fonctions
Stratégie différente	3 Récursion	Parcours récursif de la liste
	2 Compréhension	Compréhension de liste, puis `sum` et `len`
	10 Témoin négatif	`functools.reduce` avec une `lambda`

Table 5.3 – Variantes contrôlées dérivées de la correction de référence.

Catégorie	Variante	TSED	EIIA	NCST
Contrôles	11 Copie exacte	1,000	1,000	1,000
	7 Condition `== 1`	1,000	1,000	1,000
Invariance	13 Instructions inversées	0,900	1,000	1,000
	6 Opérateurs équivalents	0,902	0,979	1,000
Même stratégie	4 Condition inversée	0,854	0,952	0,815
	9 `while` avec index	0,617	0,671	0,897
	12 Sortie ajoutée	0,561	0,710	0,929
	5 Code mort	0,493	**0,541**	0,765
	8 Fonctions auxiliaires	0,425	0,924	0,774
Stratégie différente	3 Récursion	0,333	0,827	0,560
	2 Compréhension	0,175	0,711	0,632
	10 Témoin négatif	**0,075**	**0,853**	0,526

Table 5.4 – Scores obtenus entre la correction de référence `01_base.py` et chacune de ses douze variantes contrôlées.

Les deux contrôles se comportent comme prévu sur les trois mesures. La copie exacte obtient un score de 1, ce qui valide le fonctionnement du pipeline. La variante 7 obtient également 1 partout : les littéraux `0` et `1` sont tous deux remplacés par la catégorie `NUM` et l'opérateur == est inchangé. Aucune des trois mesures ne distingue donc la somme des pairs de la somme des impairs. Ce résultat illustre la différence entre équivalence fonctionnelle et substituabilité : les deux programmes ne calculent pas la même chose, mais ils mobilisent exactement la même démarche (parcours, test, double accumulation). Du point de vue pédagogique, le score de 1 est donc pertinent, même s'il serait faux pour une tâche de vérification fonctionnelle.

5.2.1 EIIA : attendus et observés

Attendus. L'EIIA compare des effectifs de nœuds. Il doit rester à 1 pour toute transformation qui ne modifie aucun effectif : renommage, changement de littéral, permutation d'instructions. Il doit diminuer dès qu'une variante ajoute ou retire des nœuds, quelle que soit la place de ces nœuds dans le programme. Comme les composantes ne sont pas pondérées, la diminution doit être d'autant plus forte que la variante modifie le nombre total de nœuds, composante dont l'échelle domine le profil. En revanche, rien ne garantit qu'une stratégie différente produise un score faible : deux programmes de taille comparable peuvent avoir des profils proches.

Observés. Les invariances sont respectées : la variante 13 obtient 1 et la variante 6 obtient 0,979, la légère baisse provenant du nœud supplémentaire introduit par les parenthèses. Le comportement sur les autres variantes confirme toutefois que l'EIIA mesure un volume plutôt qu'une organisation.

— Le score le plus faible (0,541) est attribué au code mort, alors que cette variante ne modifie en rien la stratégie de résolution. Les instructions inutiles augmentent fortement le nombre de nœuds, d'affectations, d'appels et d'itérations.

— Le témoin négatif `reduce` obtient 0,853, soit davantage que la boucle `while` (0,671) qui suit pourtant la même démarche que la référence. Le programme est court, mais l'expression de la `lambda` contient une condition, des opérateurs et des appels dont les effectifs restent proches de ceux de la référence.

— Les fonctions auxiliaires (0,924) et la récursion (0,827) restent également élevées : déplacer le code dans des fonctions ou remplacer la boucle par des appels récursifs modifie peu le volume global.

Sur les douze variantes, l'EIIA ne descend pas sous 0,54 et ne sépare pas les stratégies différentes des réécritures de même stratégie. Il doit donc être interprété comme un indicateur de taille et de complexité comparables, utile pour écarter des paires de dimensions très différentes, et non comme une mesure de proximité de démarche.

5.2.2 TSED : attendus et observés

Attendus. Grâce à l'anonymisation et à l'abstraction des littéraux, le TSED doit valoir 1 pour la copie et pour la variante 7. Il doit diminuer légèrement pour un changement d'opérateur (une substitution) et pour une permutation d'instructions, puisque la distance d'édition tient compte de l'ordre des nœuds. Toute réécriture de la boucle ou ajout d'instructions doit le faire baisser davantage, et les stratégies différentes doivent obtenir les scores les plus faibles.

Observés. Le TSED présente la plage de valeurs la plus étendue, de 0,075 à 1. Les deux contrôles obtiennent 1 et les trois stratégies différentes occupent le bas du classement, le témoin négatif obtenant le score le plus faible (0,075). C'est la seule mesure qui ordonne correctement les deux extrêmes.

Les deux variantes d'invariance perdent chacune quatre éditions :

— Pour la variante 13 ($1 - 4/40 = 0{,}900$), la permutation interagit avec l'anonymisation canonique. Les variables étant renommées dans leur ordre d'apparition, `nb` devient `VAR_1` au lieu de `VAR_2`. Les deux mises à jour de la boucle et les deux éléments du `return` diffèrent alors chacun d'un label, soit quatre substitutions. Une simple permutation se traduit donc par des changements de noms canoniques.

— Pour la variante 6 ($1 - 4/41 \approx 0{,}902$), le remplacement de `%` par `&` et le nœud des parenthèses expliquent deux éditions. **[À vérifier :** origine des deux éditions restantes, par exemple une différence entre l'expansion de += et la forme écrite explicitement**]**

Le TSED se montre en revanche sévère envers les réécritures qui conservent la démarche. La boucle `while` (0,617), la sortie ajoutée (0,561) et le code mort (0,493) perdent entre 40 et 50 % de similarité. Les fonctions auxiliaires obtiennent 0,425, un score proche de celui de la récursion (0,333). Pour cette variante, seule la fonction contenant le plus de nœuds (`stats_pairs`) est comparée, et la fonction `est_pair` n'est pas *inlinée* : l'affectation intermédiaire `n0 = n` la rend non éligible aux règles du chapitre précédent. **[À vérifier :** inlining effectif de `accumuler`**]** Le TSED distingue donc très bien les programmes très différents, mais il confond en partie une réorganisation du code avec un changement de stratégie.

5.2.3 NCST : attendus et observés

Attendus. Le NCST supprime les noms, les valeurs, les expressions et les opérateurs. Il doit donc valoir 1 pour les contrôles, pour la variante 6 et pour la variante 13, dont les instructions sont toutes des nœuds `STATEMENT`. Les réécritures de même stratégie doivent conserver le squelette `ITERATION` → `SELECTION` et n'ajouter que des nœuds : le score doit baisser modérément. Les stratégies différentes doivent faire disparaître ce squelette et obtenir les scores les plus faibles.

Observés. Les invariances sont toutes respectées (1 pour les variantes 6, 7, 11 et 13). Le NCST de la référence compte $N_s = 13$ nœuds et correspond exactement à l'exemple de la figure 4.8. Les scores des réécritures de même stratégie s'expliquent directement par la formule de Sørensen-Dice :

— boucle `while` : trois instructions supplémentaires (initialisation de l'indice, lecture de l'élément, incrément), $N_g = 16$, $M = 13$, d'où $S = 26/29 \approx 0{,}897$;

— sortie ajoutée : deux instructions supplémentaires, $N_g = 15$, $M = 13$, d'où $S = 26/28 \approx 0{,}929$;

— code mort : huit nœuds supplémentaires, $N_g = 21$, $M = 13$, d'où $S = 26/34 \approx 0{,}765$.

Dans ces trois cas, tous les nœuds de la référence sont appariés : la baisse provient uniquement de la différence de taille. Le NCST reconnaît donc la démarche commune et reste la mesure la plus élevée pour ces variantes.

La variante 4 montre en revanche la sensibilité au niveau structurel. Le `continue` devient un nœud `TRANSFER` dans la sélection, et les deux mises à jour sortent du bloc `THEN` : leur niveau passe de 4 à 3. Ces deux nœuds ne trouvent plus de correspondant ($M = 11$, $N_g = 14$), d'où $S = 22/27 \approx 0{,}815$. Une simple inversion de condition est ainsi pénalisée davantage que l'ajout de trois instructions.

Pour les stratégies différentes, le score observé reste supérieur à 0,5. Le cas du témoin négatif est révélateur : son NCST ne compte que six nœuds (`PROGRAM`, deux `SEQUENCE`, l'import, `FUNCTION` et le

`TRANSFER` du `return`), dont cinq sont appariés avec la référence, d'où $S = 10/19 \approx 0{,}526$. La `lambda` et sa condition, contenues dans une expression, ne produisent aucun nœud. Il en va de même de la compréhension de liste de la variante 2, réduite à un `STATEMENT` (0,632). **[À vérifier :** absence des nœuds `LAMBDA` et `COMPREHENSION` lorsqu'ils figurent dans une expression**]**

Ces cas mettent en évidence un **effet plancher**. Tout programme constitué d'une fonction qui se termine par un `return` partage avec la référence au moins l'échafaudage `PROGRAM`, `SEQUENCE`, `FUNCTION`, `SEQUENCE`, `TRANSFER`, soit cinq nœuds appariés. Le score NCST d'une paire sans aucune structure de contrôle commune reste donc proche de 0,5. Les valeurs doivent être lues de manière relative : un score de 0,53 signale ici une organisation très différente, et non une similarité de moitié.

5.2.4 Bilan sur les variantes contrôlées

Les trois mesures respectent les invariances qui découlent de leurs règles de normalisation, et aucune ne distingue un changement de valeur à l'intérieur d'un test (variante 7). Elles diffèrent en revanche nettement sur les réécritures :

- le **TSED** est le plus discriminant et classe correctement les stratégies différentes, mais il pénalise fortement les réécritures de même stratégie ;
- le **NCST** est le plus robuste à ces réécritures, mais il est sensible aux changements de niveau et présente un plancher proche de 0,5 ;
- l'**EIIA** reflète la taille et le volume des programmes : il est plus sensible au code mort qu'à un changement de stratégie.

Les mesures ne sont donc pas redondantes : chacune réagit à un type de transformation différent. Ces observations justifient l'étude de leur combinaison, présentée dans la section suivante, ainsi que des améliorations possibles de la normalisation, comme la suppression du code mort, discutées dans le chapitre consacré aux limites et aux perspectives.

5.2.5 Fusion des trois approches

[À compléter : scores pondérés sur les variantes**]**

5.3 Résultats sur le corpus complet

5.4 Synthèse comparative

Chapitre 6

Discussion sur les limites et perspectives

6.1 Discussion des résultats

6.2 Limites techniques des méthodes

Conversion du CST en AST. La règle de suppression des nœuds non reconnus (section 4.3.1) garantit un AST sans étiquette artificielle, mais elle a trois contreparties.

- **Perte de la forme des constructions absentes de la table.** Seul le contenu d'une construction non reconnue reste visible. Pour une fonction décorée, l'AST ne sait plus que l'identifiant `cache` était un décorateur ; pour une instruction `match`, les différents cas ne sont plus délimités. Ce choix reste acceptable tant que ces constructions sont rares dans les corrections d'exercices, ce qui est le cas des jeux de données utilisés.
- **Perte du regroupement.** Lorsque plusieurs nœuds intermédiaires disparaissent, leurs contenus se retrouvent au même niveau. Dans l'exemple de la figure 4.3, rien n'indique plus que `fh` est associé au premier appel à `open`. Le TSED peut alors apparier des nœuds qui appartenaient à des groupes différents. C'est la raison pour laquelle les constructions porteuses d'une stratégie, comme les compréhensions, reçoivent un type propre plutôt que de relever de cette règle.
- **Suppression silencieuse.** Un type absent de la table disparaît sans signalement. La table dépend des noms de types de la grammaire Tree-sitter : si une version ultérieure de cette grammaire renomme un type, celui-ci sera supprimé sans erreur, et seule une inspection des AST produits permettra de le détecter. La table doit donc être revue à chaque changement de version de l'analyseur ou lors du passage à un autre langage.

6.3 Limites de l'évaluation

6.4 Difficultés rencontrées

6.5 perspective

Conclusion générale

[À compléter : 3 pages**]**

Bibliographie

[1] Uri Alon, Shaked Brody, Omer Levy et Eran Yahav. *code2seq : Generating Sequences from Structured Representations of Code*. 21 fév. 2019. doi : 10.48550/arXiv.1808.01400. arXiv : 1808.01400[cs.LG]. url : http://arxiv.org/abs/1808.01400.

[2] Uri Alon, Meital Zilberstein, Omer Levy et Eran Yahav. « code2vec : Learning Distributed Representations of Code ». In : *Proceedings of the ACM on Programming Languages (POPL)* (2019).

[3] I.D. Baxter et al. « Clone detection using abstract syntax trees ». In : *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. Proceedings. International Conference on Software Maintenance. Bethesda, MD, USA : IEEE Comput. Soc, 1998, p. 368-377. isbn : 978-0-8186-8779-2. doi : 10.1109/ICSM.1998.738528. url : http://ieeexplore.ieee.org/document/738528/.

[4] Nghi D. Q. Bui, Yijun Yu et Lingxiao Jiang. « Self-Supervised Contrastive Learning for Code Retrieval and Summarization via Semantic-Preserving Transformations ». In : *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2021, p. 511-521.

[5] Ting Chen, Simon Kornblith, Mohammad Norouzi et Geoffrey Hinton. « A Simple Framework for Contrastive Learning of Visual Representations ». In : *Proceedings of the 37th International Conference on Machine Learning (ICML)*. T. 119. Proceedings of Machine Learning Research. 2020, p. 1597-1607.

[6] Alisson Antônio De Oliveira. « Assessing Programming Difficulty and Effort : Statistical Correlations with the Index of Internal Effort ». In : *Anais da XII Escola Regional de Informática de Goiás (ERI-GO 2024)*. Escola Regional de Informática de Goiás. Brasil : Sociedade Brasileira de Computação, 5 déc. 2024, p. 21-30. doi : 10.5753/erigo.2024.4795. url : https://sol.sbc.org.br/index.php/erigo/article/view/32208.

[7] Zhangyin Feng, Daya Guo, Duyu Tang et al. « CodeBERT : A Pre-Trained Model for Programming and Natural Languages ». In : *Findings of EMNLP*. 2020, p. 1536-1547.

[8] Tianyu Gao, Xingcheng Yao et Danqi Chen. « SimCSE : Simple Contrastive Learning of Sentence Embeddings ». In : *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021, p. 6894-6910.

[9] Daya Guo, Shuo Ren, Shuai Lu et al. « GraphCodeBERT : Pre-training Code Representations with Data Flow ». In : *ICLR*. 2021.

[10] Keith J. Holyoak et Kyunghee Koh. « Surface and structural similarity in analogical transfer ». In : *Memory & Cognition* 15.4 (juill. 1987), p. 332-340. issn : 0090-502X, 1532-5946. doi : 10.3758/BF03197035. url : http://link.springer.com/10.3758/BF03197035.

[11] Roya Hosseini et Peter Brusilovsky. « A study of concept-based similarity approaches for recommending program examples ». In : *New Review of Hypermedia and Multimedia* 23.3 (2017), p. 161-188.

[12] Paras Jain et al. « Contrastive Code Representation Learning ». In : *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021, p. 5954-5971.

[13] Sébastien Jolivet, Badmavasan Kirouchenassamy, Amel Yessad et Vanda Luengo. « Modélisation et décision de rétroactions épistémiques dans l'environnement AlgoPython ». In : *Environnements Informatiques pour l'Apprentissage Humain*. Lille, France : Université de Lille, juin 2025. url : https://hal.science/hal-05163937.

[14] Qi Liu et al. « Finding Similar Exercises in Online Education Systems ». In : *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2018, p. 1821-1830.

[15] Yiyang Lu et al. *CSSG : Measuring Code Similarity with Semantic Graphs*. 14 jan. 2026. doi : 10.48550/arXiv.2601.04085. arXiv : 2601.04085[cs.PL]. url : http://arxiv.org/abs/2601.04085.

[16] Marcus Messer, Neil C. C. Brown, Michael Kölling et Miaojing Shi. « Automated Grading and Feedback Tools for Programming Education : A Systematic Review ». In : *ACM Transactions on Computing Education* 24.1 (2024).

[17] Dragana Milovancevic. « Automated Induction for Proving Program Equivalence ». In : ().

[18] Dragana Milovančević et al. « Formal Autograding in a Classroom ». In : *Programming Languages and Systems*. Sous la dir. de Viktor Vafeiadis. T. 15695. Series Title : Lecture Notes in Computer Science. Cham : Springer Nature Switzerland, 2025, p. 154-174. isbn : 978-3-031-91120-0 978-3-031-91121-7. doi : 10.1007/978-3-031-91121-7_7. url : https://link.springer.com/10.1007/978-3-031-91121-7_7.

[19] Jie Mu et al. « BCRNet : Bidirectional contrastive representation network for deep multimodal learning of exercise representations ». In : *Neurocomputing* 548 (2023), p. 126409.

[20] Aaron van den Oord, Yazhe Li et Oriol Vinyals. *Representation Learning with Contrastive Predictive Coding*. 2018. arXiv : 1807.03748.

[21] José Paiva, José Leal et Álvaro Figueira. « Comparing semantic graph representations of source code : The case of automatic feedback on programming assignments ». In : *Computer Science and Information Systems* 21.1 (2024), p. 117-142. issn : 1820-0214, 2406-1018. doi : 10.2298/CSIS230615004P. url : https://doiserbia.nb.rs/Article.aspx?ID=1820-02142400004P.

[22] Radek Pelánek. « Measuring Similarity of Educational Items : An Overview ». In : *IEEE Transactions on Learning Technologies* 13.2 (2020), p. 354-366.

[23] Lutz Prechelt, Guido Malpohl et Michael Philippsen. « Finding Plagiarisms among a Set of Programs with JPlag ». In : ().

[24] Manda Uday Kiran Reddy, Shreyas Pujar et Srujana Inturi. « Graph-Based Semantic Similarity for Programming Exercise Assessment : A Survey ». In : *2025 5th International Conference on Intelligent Technologies (CONIT)*. 2025 5th International Conference on Intelligent Technologies (CONIT). Juin 2025, p. 1-8. doi : 10.1109/CONIT65521.2025.11167193. url : https://ieeexplore.ieee.org/abstract/document/11167193.

[25] Nils Reimers et Iryna Gurevych. « Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks ». In : *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Hong Kong, China : Association for Computational Linguistics, 2019, p. 3982-3992. doi : 10.18653/v1/D19-1410. url : https://www.aclweb.org/anthology/D19-1410.

[26] Yewei Song et al. « Revisiting Code Similarity Evaluation with Abstract Syntax Tree Edit Distance ». In : *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2 : Short Papers)*. 2024, p. 38-46. doi : 10.18653/v1/2024.acl-short.3. arXiv : 2404.08817[cs.CL]. url : http://arxiv.org/abs/2404.08817.

[27] Zejie Tian, Brendan Flanagan, Yiling Dai et Hiroaki Ogata. « Automated Matching of Exercises with Knowledge Components ». In : *Proceedings of the 30th International Conference on Computers in Education (ICCE)*. 2022, p. 24-33.

[28] Milena Vujošević-Janičić, Mladen Nikolić, Dušan Tošić et Viktor Kuncak. « Software verification and graph similarity for automated evaluation of students' assignments ». In : *Information and Software Technology* 55.6 (juin 2013), p. 1004-1016. issn : 09505849. doi : 10.1016/j.infsof.2012.12.005. url : https://linkinghub.elsevier.com/retrieve/pii/S0950584912002406.

[29] Tiantian Wang, Kechao Wang, Xiaohong Su et Peijun Ma. « Detection of semantically similar code ». In : *Frontiers of Computer Science* 8.6 (1er déc. 2014), p. 996-1011. issn : 2095-2236. doi : 10.1007/s11704-014-3430-1. url : https://doi.org/10.1007/s11704-014-3430-1.

[30] Chengguan Xiang, Ying Wang, Qiyun Zhou et Zhen Yu. « Graph semantic similarity-based automatic assessment for programming exercises ». In : *Scientific Reports* 14.1 (8 mai 2024), p. 10530. issn : 2045-2322. doi : 10.1038/s41598-024-61219-8. url : https://www.nature.com/articles/s41598-024-61219-8.

[31] Kaizhong Zhang et Dennis Shasha. « Simple Fast Algorithms for the Editing Distance between Trees and Related Problems ». In : *SIAM Journal on Computing* 18.6 (déc. 1989), p. 1245-1262. issn : 0097-5397, 1095-7111. doi : 10.1137/0218082. url : http://epubs.siam.org/doi/10.1137/0218082.

[32] Adel Ziani. *Détection automatique d'exercices de programmation similaires*. Rapport de stage de M1 informatique, laboratoire Dicen, Université Gustave Eiffel. 2025.

Annexe A

Annexes

[**À compléter :** codes des variantes, tableaux complets, scripts, résultats bruts]